

Laphorn: Exact-Marginal Neural Layers Without Iterative Normalisation

Łukasz Struski Bartosz Wójcik Marcin Sendera Jacek Tabor

August 21, 2026

Abstract

Balanced attention, routing, and mixing layers promise neural networks that move information while respecting global capacity or conservation constraints. The standard way to turn learned scores into a matrix with prescribed row and column sums, however, inserts an iterative Sinkhorn loop into every forward and backward pass. We show that this loop is unnecessary when the network needs a learnable feasible coupling rather than the minimiser of a specified optimal-transport objective. LAPHORN maps arbitrary scores directly to an exact-marginal coupling by a fixed, non-iterative computation: a softmax and target row scale satisfy one marginal, and one differentiable quantile-barrier scan satisfies the other. The layer can return the full coupling, or apply it directly to value channels without materialising either the quantile strip or the coupling. On a presorted grid, full construction takes $\mathcal{O}(nm)$ work; direct application takes $\mathcal{O}(nm + nmd)$ work and only $\mathcal{O}((n + m)d)$ additional memory beyond the dense score weights, with no convergence-dependent factor. The family reaches every interior coupling at all sufficiently small positive widths and is dense in the full transportation polytope. Its backward is an explicit scan-based vector–Jacobian product, not an unrolled normaliser or fixed-point solve. A deterministic fp64 audit verifies feasibility and gradients at numerical precision. On an RTX 5060, direct apply is $5.1\times$ faster in inference and uses $3.0\times$ less incremental peak training memory at $n = 4096$ than the best measured path that builds a fresh plan. Across calibrated Sinkhorn+ X comparisons, even the lower-memory Sinkhorn read-out requires $1.15\text{--}2.40\times$ its training peak. Compiled Sinkhorn remains faster for an easy $\varepsilon = 1$ kernel, whereas compiled LAPHORN is $6.9\times$ faster at $n = 1024$ for $\varepsilon = 0.3$ and up to $32.3\times$ faster for complete multi-channel application at $n = 4096$. In controlled derivative tests, Sinkhorn required 61–108 unrolls before truncation error fell below LAPHORN’s finite-difference verification threshold. LAPHORN turns exact marginal constraints from an iterative projection into a practical differentiable layer with both matrix and matrix-free interfaces.

1 Introduction

Neural networks increasingly use matrices not merely as weights, but as structured communication patterns. A balanced attention map can prevent all mass from collapsing onto a few tokens; a routing matrix can respect expert capacity; a mixing layer can preserve a prescribed amount of signal in every source and destination. These examples share the same mathematical interface: a network predicts scores, and the layer must turn them into a nonnegative matrix Π with specified row and column sums. Such matrices already appear in attention, expert routing, soft permutations, matching, and transport [26, 28, 20, 30, 22]. If this constraint were as cheap and predictable as a softmax, exact-marginal matrices could become ordinary building blocks rather than occasional optimisation subroutines.

The usual obstacle is the normalisation loop. Sinkhorn–Knopp starts from a positive score kernel and alternately rescales its rows and columns until both marginals meet a tolerance [8]. This is an elegant and general solver, especially when the desired output is an entropically regularised optimal-transport plan. Inside a neural network, however, the iteration count becomes part of the layer: it affects latency, memory, compilation, and the gradient. Recent architectures invoke such normalisation in every layer or optimisation step; mHC, for example, uses roughly 20 iterations for constrained residual mixing [24], and routing and doubly-stochastic transformer work report the loop as a practical overhead [27, 25]. Moreover, a small forward marginal residual does not guarantee that an unrolled derivative is close to the derivative of the converged map, while implicit differentiation adds a linear solve after convergence.

This paper asks a layer-design question rather than an optimal-transport question: *can arbitrary scores become an expressive exact-marginal matrix through a fixed non-iterative computation?* LAPHORN answers yes. Its central observation is simple. A row-wise softmax W already satisfies $W\mathbf{1} = \mathbf{1}$, so

multiplying row i by the target mass s_i fixes one marginal immediately. Only the column mass $c = W^\top s$ remains. One differentiable quantile construction partitions its smoothed cumulative distribution into target masses t_j ; the resulting strip B redistributes the columns once. The output

$$\Pi = \text{diag}(s) W B^\top \quad (1)$$

has row sums s and column sums t for every score matrix and every positive width. Exact feasibility is therefore a property of the construction, not the stopping condition of a solver. Throughout, *one-pass* means this fixed, non-iterative sequence; it does not mean one kernel launch or subquadratic dense attention.

The layer is useful in two different ways. When a downstream module needs the matrix, LAPHORN returns Π directly. More often, attention, routing, and feature transport only need the action $Y = \Pi X$ on d value channels. In that case associativity exposes a stronger interface:

$$(\ell, X) \mapsto Y = \text{diag}(s) W (B^\top X). \quad (2)$$

The computation first contracts the narrow value matrix with the strip by a scan, then performs one dense multiply by W , and finally scales the output rows. Neither B nor Π is formed. On the default presorted grid, score preprocessing and full coupling construction require $\mathcal{O}(nm)$ work. Direct application requires $\mathcal{O}(nm + nmd)$ work and $\mathcal{O}((n+m)d)$ additional memory beyond W . By contrast, an L -iteration Sinkhorn layer requires $\mathcal{O}(Lnm)$ scaling work before the shared $\mathcal{O}(nmd)$ application. Sinkhorn can also apply its factorised plan without materialising Π , but only after the scaling vectors u, v have been obtained to a requested feasibility tolerance. Thus the advantage of LAPHORN is not sparse or subquadratic attention; it is predictable exact feasibility, no convergence loop, and a direct score-to-output path that avoids unnecessary intermediate matrices.

The fixed computation does not sacrifice the geometry of the constraint. Every interior point of the rectangular transportation polytope is represented exactly at all sufficiently small positive widths, and the closure of the family is the full polytope, including scaled permutations in the square-uniform case. The reverse pass is equally direct: an explicit matrix-free vector–Jacobian product reuses the forward scans, with no unrolled normaliser and no fixed-point linear solve. Pure-PyTorch, `torch.compile`, and fused-CUDA implementations expose both the full-coupling and direct-apply interfaces.

The scope is deliberately different from solving optimal transport. If a prescribed cost should select a particular entropically regularised minimiser, Sinkhorn is the appropriate tool. LAPHORN targets the common neural-layer setting in which learned scores should parameterise *some* expressive differentiable coupling with exact marginals. Its strip also uses an order along one axis; this is natural for tokens, ranks, and spatial positions, but must be chosen or learned for an unordered set.

The measurements show why this distinction matters in practice. A deterministic fp64 audit validates exact marginals, agreement of both interfaces, and the analytic derivative. Compiled Sinkhorn wins for an easy kernel that converges in three iterations, while LAPHORN is $6.9\times$ faster at $n = 1024$ in the sharper $\varepsilon = 0.3$ regime. For complete score-to-multi-channel-output computation, the advantage reaches $32.3\times$ at $n = 4096$. Against constructing and immediately applying its own full coupling, direct apply is $5.1\times$ faster in inference and cuts incremental peak training memory from 578 to 194 MiB at $n = 4096$. Against accuracy-calibrated Sinkhorn followed by X , its training peak is also smaller in every measured case: the better Sinkhorn read-out requires $1.15\text{--}2.40\times$ as much memory. Derivative-controlled tests further show that 61–108 Sinkhorn unrolls are required before truncation error falls below LAPHORN’s finite-difference verification threshold. These are layer-level systems results, not claims that the two methods optimise the same objective.

Contributions.

- **An exact-marginal neural layer without iterative normalisation.** A weighted, multi-level barrier map turns arbitrary scores into rectangular couplings with non-uniform prescribed marginals in one fixed computation (Proposition 1).
- **A direct score-to-output interface.** When only $Y = \Pi X$ is needed, LAPHORN applies the latent coupling without forming either the strip or Π . This is a first-class layer operation, not a multiplication performed after plan construction.
- **Expressivity and a direct backward.** Every interior coupling is attained exactly at all sufficiently small positive widths and the family is dense in the full transportation polytope (Theorems 1–2). Reverse mode is an explicit matrix-free scan-based VJP rather than a differentiated fixed point.

- **Practical implementations and controlled evidence.** Pure-PyTorch, `torch.compile`, and fused-CUDA paths cover both interfaces. The numerical audit verifies feasibility and gradients, while feasibility- and derivative-controlled comparisons identify both the easy regime won by Sinkhorn and the sharper regimes where avoiding iteration yields up to a $32.3\times$ end-to-end advantage. A six-path GPU audit also shows that direct apply cuts peak training memory by $3.0\times$ against the best measured fresh-plan path at $n = 4096$; a separate Sinkhorn+X audit gives a $1.15\text{--}2.40\times$ training-memory advantage.

We first position the method relative to Sinkhorn, exact polytope charts, and differentiable order operators (§2.1). Sections 3 and 4 develop the barrier primitive and the coupling map; §5 evaluates its numerical accuracy and cost.

2 Related Work and Background

2.1 Couplings with prescribed marginals

Given positive row totals $s \in \mathbb{R}^n$ and column totals $t \in \mathbb{R}^m$ with equal total mass, the transportation polytope is

$$U(s, t) = \{\Pi \in \mathbb{R}_{\geq 0}^{n \times m} : \Pi \mathbf{1} = s, \Pi^\top \mathbf{1} = t\}.$$

We use the unit-mass convention $\sum_i s_i = \sum_j t_j = 1$ throughout. Thus, in the square uniform case, $s = t = \mathbf{1}/n$ and $n\Pi$ is doubly stochastic in the standard convention (row and column sums equal to one). This explicit scaling avoids conflating probability couplings with attention matrices.

Sinkhorn–Knopp scaling starts from a positive kernel K and alternates row and column rescalings until the prescribed marginals hold [9, 10]. For a cost matrix, $K = \exp(-C/\varepsilon)$ gives entropic optimal transport [8]; for learned scores, the same operation is used as a differentiable normalisation layer [15, 20]. After convergence the plan factors as $\Pi = \text{diag}(u)K\text{diag}(v)$, so its action can be evaluated without materialising an additional copy of Π . The unavoidable cost is instead the sequence of dense kernel-vector products required to obtain u and v .

Exact parametrisations of doubly-stochastic matrices. The Birkhoff–von Neumann theorem represents a doubly-stochastic matrix as a convex combination of permutations, but the corresponding simplex has $n!$ vertices [11]. More compact maps include the differentiable stick-breaking chart of Linderman et al. [12] and recent transportation-polytope charts for manifold-constrained hyper-connections [13]. These are natural baselines when the goal is a free-standing coordinate system for the square uniform polytope. Our setting is different: an arbitrary score matrix is read into a rectangular coupling with possibly non-uniform marginals, and the read-out must support a batched reverse pass. We therefore compare with charts only on their overlapping square-uniform regime and treat Sinkhorn as the primary score-conditioned baseline.

Differentiating Sinkhorn. The common derivative unrolls L scaling steps and differentiates the truncated computation [15, 20]. This returns the gradient of the L -step map and stores or recomputes its trajectory. Implicit differentiation instead solves a linear system at the converged fixed point. Both approaches can differentiate with respect to scores, marginals, and regularisation parameters; the distinction relevant here is computational, not one of formal differentiability. LAPHORN replaces the fixed point by an explicit score-to-coupling map whose VJP is obtained from a small number of structured scans.

2.2 Background: the LapSum operator

LapSum [5] expresses soft ranking, top- k , and sorting as readings of a smoothed cumulative function. For scores $r_1, \dots, r_n \in \mathbb{R}$ and width $\alpha \neq 0$, it defines

$$\text{LapSum}_\alpha(r, x) = \sum_{i=1}^n \sigma\left(\frac{x - r_i}{\alpha}\right), \quad \sigma(u) = \begin{cases} \frac{1}{2}e^u, & u \leq 0, \\ 1 - \frac{1}{2}e^{-u}, & u > 0, \end{cases} \quad (3)$$

an increasing function that approaches a step count as $\alpha \rightarrow 0$. Soft top- k , for example, uses the threshold $b = \text{LapSum}_\alpha^{-1}(r, k)$ and the mask $p_i = \sigma((b - r_i)/\alpha)$, whose entries sum exactly to k . On a sorted axis,

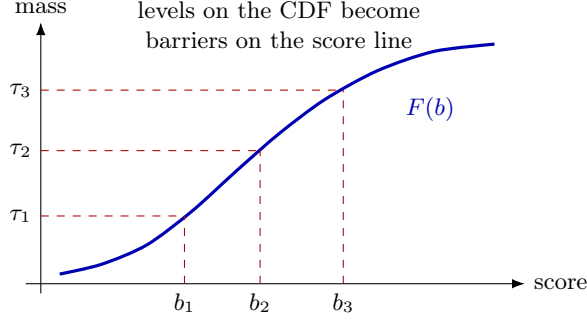


Figure 1: LapSum barriers are inverse-CDF values. Prescribed mass levels τ_i on the vertical axis determine score locations $b_i = F^{-1}(\tau_i)$. The rest of the paper uses these barriers as thresholds, bin edges and coupling boundaries.

prefix and suffix recurrences reduce each inverse query to a per-interval quadratic; the forward and its implicit VJP cost $\mathcal{O}(n \log n)$ time and $\mathcal{O}(n)$ memory (Appendix B).

LapSum uses unweighted points and either one target level or a fixed arithmetic grid. We extend its inverse to weighted points and an arbitrary vector of target levels. This weighted, many-level inverse is the barrier map developed next; from here on its width is denoted by h .

3 Many barriers as a primitive

Everything that follows starts with one operation: invert a smoothed score distribution at many prescribed mass levels, placing a differentiable barrier at each. The target levels live on the vertical axis of a CDF; the barriers are the corresponding locations on the score axis. Figure 1 is the picture to keep in mind. Once these cuts are available, they can be read as quantiles, selection thresholds, bins for a coupling, or bins for a transport plan. The important computational point is that a whole vector of cuts is still one parallel primitive: one sorted scan, many independent inverse-CDF queries, and the same scan again in reverse.

Take points $x_1, \dots, x_n$ (the token coordinates), per-token weights $w_j \geq 0$, $\sum_j w_j = 1$ — either uniform ($w_j = 1/n$) or a softmax of input logits, in which case we carry $\log w_j = \text{logsoftmax}$ for stability — and a width $h > 0$. These define the kernel-smoothed weighted CDF

$$F(b) = \sum_{j=1}^n w_j \sigma\left(\frac{b - x_j}{h}\right), \quad (4)$$

a continuous increasing bijection $\mathbb{R} \rightarrow (0, 1)$. Given any set of target levels $\tau_1, \dots, \tau_m \in (0, 1)$ — themselves supplied through their logits for stability — the *barriers* are

$$b_i = F^{-1}(\tau_i), \quad i = 1, \dots, m. \quad (5)$$

They are obtained by the closed-form inverse of §2.2: one shared sort of the scores, followed by m inverse-CDF queries. Thus asking for many barriers is not a new dense operation; it is the same scan-based primitive used repeatedly on different mass levels.

The barrier Jacobian is dense as a mathematical object. At million-token scale, even a moderate number of barriers would make it impossible to store. The key point is that reverse mode never needs this Jacobian; it only needs its vector–Jacobian product. If $K_f[i, j] = f((b_i - x_j)/h)$ is the Laplace-density kernel and $K_\sigma[i, j] = \sigma((b_i - x_j)/h)$ is the CDF kernel, the VJP has the form

$$\frac{\partial L}{\partial x} = w \odot (K_f^\top g), \quad \frac{\partial L}{\partial \log w} = -h w \odot (K_\sigma^\top g), \quad d = K_f w, \quad (6)$$

for a vector g obtained from the upstream cotangent. Each multiply is a one-dimensional Laplace-kernel scan, so the backward remains matrix-free and exposes the same batched parallelism as the forward. This is not just a memory saving: it is why the barrier map is useful as a layer. The analytic gradient is a fast tensor computation, not a dense Jacobian, not an unrolled solver, and not an implicit linear

system. The full derivation, including the implicit derivatives

$$\frac{\partial b_i}{\partial x_j} = \frac{w_j f((b_i - x_j)/h)}{d_i}, \quad d_i = \sum_l w_l f((b_i - x_l)/h), \quad f(u) = \frac{1}{2}e^{-|u|}, \quad (7)$$

is in Appendix A. The appendix also shows how the density d_i is read from the forward inverse, so the backward does not add another full sort.

In applications the mass levels usually come from logits, not from exact probabilities. We therefore carry the construction in the log/logit domain: weights use $\log w$, levels use logits of τ_i , and interval masses are computed without subtracting nearly equal CDF values. This keeps peaked top- k -like regimes stable and lets autograd reproduce the implicit barrier gradient exactly (Appendix A).

Read at the appropriate weights and levels, the barrier map immediately reproduces LapSum’s differentiable order operators — soft rank, budgeted top- k and the doubly-stochastic soft permutation among them — each a different reading of the same sorted pass, and in the zero-width equal-mass limit realising *every* permutation (Appendix C, Theorem 3). These recover what LapSum already provided; the rest of the paper develops what it did not — the explicit quantile-barrier construction for the transportation polytope (§4) and its exact, matrix-free gradient.

4 A quantile-barrier map into the transportation polytope

This section builds the map at the centre of the paper: an explicit, closed-form read-out that lands in the transportation polytope $U(s, t)$ in a single pass, using only the barrier map of §3. The construction is *directional*: a row-wise softmax makes one marginal correct *for free*, a plain diagonal scales it to s , and a *single* barrier strip reconciles the other marginal — one softmax, one barrier solve, one strip. It holds both marginals *exactly* for any input, attains every interior point at all sufficiently small positive widths, and has the full polytope as its joint positive-weight/zero-width closure. The scores, width, and marginals are differentiable inputs. The width h controls smoothing, but unlike the entropic parameter in an OT solve it does not trade marginal feasibility for regularity: both marginals are exact for every $h > 0$. In the square uniform case, rescaling by n gives a standard doubly-stochastic layer; the same construction handles rectangular and non-uniform marginals.

The construction: one marginal free, one reconciled. Fix marginals $s \in \Delta^{n-1}$, $t \in \Delta^{m-1}$, the cumulative levels $T_j = \sum_{j' \leq j} t_{j'}$, and write $U(s, t) = \{\Pi \in \mathbb{R}_{\geq 0}^{n \times m} : \Pi \mathbf{1} = s, \Pi^\top \mathbf{1} = t\}$ for the transportation polytope of §2.1. From a score (logit) matrix $\ell \in \mathbb{R}^{n \times m}$, take a *row-wise* softmax

$$W = \text{softmax}(\ell, \text{axis}=2), \quad W \mathbf{1} = \mathbf{1}, \quad (8)$$

so each row of W is a distribution over the columns — the soft assignment attention already forms. *One marginal is now free of charge*: because every row of W sums to one, scaling row i by s_i makes the row sums *exactly* s , so $\text{diag}(s)W$ fixes the row marginal with *no* cut — just a diagonal. Only the *column* marginal remains. Lay the column indices on a line $\eta_1 < \dots < \eta_m$ (the integer grid $0, \dots, m-1$ in the default layer), let $c = W^\top s$ be the column mass that $\text{diag}(s)W$ currently carries, and solve *one* barrier map (§3) on that axis. Set $\beta_0 = -\infty$, $\beta_m = +\infty$, and, for $j = 1, \dots, m-1$, let $\beta_j = F_Y^{-1}(T_j)$, where $F_Y = \sum_{j'} c_{j'} \sigma\left(\frac{\cdot - \eta_{j'}}{h}\right)$ is the smoothed column CDF. Thus only the $m-1$ finite internal cuts are solved. Redistributing each column’s mass onto those cuts is a single *strip*

$$B = B_h(c), \quad B_{jj'} = \sigma\left(\frac{\beta_j - \eta_{j'}}{h}\right) - \sigma\left(\frac{\beta_{j-1} - \eta_{j'}}{h}\right), \quad (9)$$

and the coupling is the product

$$\Pi(\ell, h) = \text{diag}(s) W B^\top. \quad (10)$$

The row marginal is exact by the diagonal and the column marginal exact by the one strip, in a *single* barrier solve. Because the strip preserves row sums (Lemma 1 below), reconciling the columns leaves the free row marginal untouched, so one pass suffices — no rescaling loop, no per-entry feasibility recursion, no entropic fixed point.

Proposition 1 (Exact marginals). *For any row-stochastic W ($W \mathbf{1} = \mathbf{1}$) and any $h > 0$, with $B = B_h(c)$ the column strip (9) for the mass $c = W^\top s$, the coupling $\Pi = \text{diag}(s) W B^\top$ of (10) lies in $U(s, t)$.*

row-wise softmax scaled by s + one column-quantile strip = coupling in $U(s, t)$

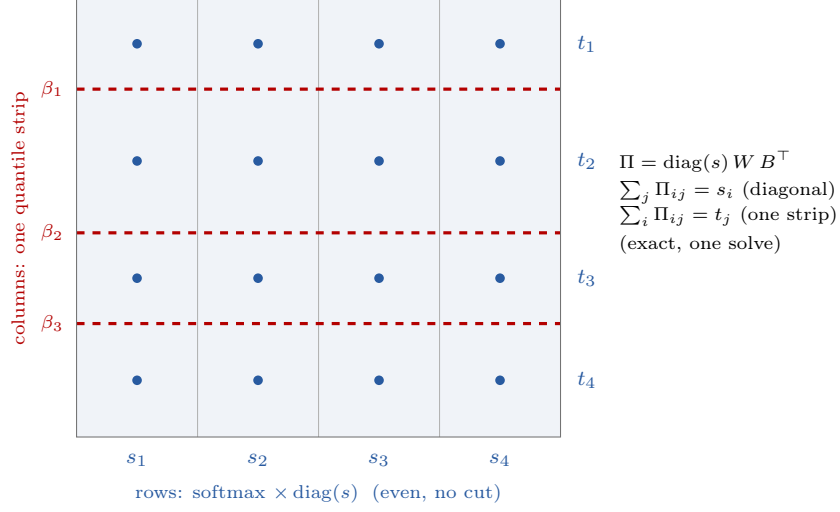


Figure 2: LAPHORN in one picture. A *row-wise* softmax turns each row of scores into a distribution over the columns (dots), so every row already sums to one; scaling row i by s_i then makes the row masses *exactly* s — a plain diagonal, no cut (even grey dividers). Only the columns remain, and a *single strip* fixes them: the column axis is cut at the *quantiles* β of the column masses (red, uneven), returned by the barrier map, making the column sums *exactly* t . The result $\Pi = \text{diag}(s) W B^\top \in U(s, t)$ holds in one pass and *one* barrier solve (no Sinkhorn iteration, no entropic ϵ); decreasing h localises redistribution near the quantile cuts, while increasing it smooths redistribution along the ordered axis.

Proof. By Lemma 1, B is column-stochastic ($B^\top \mathbf{1} = \mathbf{1}$) and obeys the marginal identity $Bc = t$. Applying $B^\top$ on the right preserves row sums, so $\Pi \mathbf{1} = \text{diag}(s) W B^\top \mathbf{1} = \text{diag}(s) W \mathbf{1} = \text{diag}(s) \mathbf{1} = s$; and $\Pi^\top \mathbf{1} = B W^\top \text{diag}(s) \mathbf{1} = B W^\top s = Bc = t$. $\square$

Choosing the softmax *is* choosing the coupling: the diagonal pins the row marginal and the one strip pins the column marginal automatically, whatever ℓ is — no rescaling and no fixed point.

Why the strip is exact, and why the family is complete. The whole construction rests on one object — the column strip $B_h(c)$, which moves the column masses onto their quantile cuts. It has exactly the two properties the marginals need, and a third that gives completeness.

Lemma 1 (The column strip). *Fix the column coordinates $\eta_1 < \dots < \eta_m$ with minimal gap $g > 0$. For any mass $c \in \Delta^{m-1}$, set $\beta_0 = -\infty$, $\beta_m = +\infty$, let the internal β_j be the T_j -quantiles of $F_Y = \sum_{j'} c_{j'} \sigma(\frac{\cdot - \eta_{j'}}{h})$, and let $B_h(c)$ be the strip (9). Then $B_h(c)$ is column-stochastic ($\mathbf{1}^\top B_h(c) = \mathbf{1}^\top$) and obeys the marginal identity $B_h(c)c = t$. Moreover there are $\rho, C, h_1 > 0$ (depending on g and $\min_j t_j$) with $\|B_h(t) - I\|_\infty \leq Ce^{-\rho/h} < 1$ for all $h \leq h_1$; in particular $B_h(t)$ is invertible there (Appendix D).*

These are precisely the facts Proposition 1 used: column-stochasticity makes $B^\top$ preserve row sums, so the diagonal's row marginal survives untouched, and the marginal identity $B_h(c)c = t$ makes the column sums land on t . The third fact, $B_h(t) \rightarrow I$ as $h \rightarrow 0$, is what lets the family reach *every* coupling, not merely meet the marginals.

Theorem 1 (Completeness). *The directional family is complete: the closure of $\{\Pi(\ell, h) : \ell \in \mathbb{R}^{n \times m}, h > 0\}$ is exactly $U(s, t)$. In the square-uniform case this includes every scaled permutation coupling. This includes boundary couplings with zeros, although a finite softmax is strictly positive.*

Proof. Set $W^* = \text{diag}(s)^{-1} \Pi^*$. It is non-negative and row-stochastic, and $W^{*\top} s = t$, but it need not be the softmax of finite logits when Π^* has zeros. For $\delta \in (0, 1)$ define instead

$$W_\delta = (1 - \delta)W^* + \delta \mathbf{1}t^\top.$$

Because $t > 0$, W_δ is strictly positive, row-stochastic, and $W_\delta^\top s = (1 - \delta)t + \delta t = t$. Hence the finite logits $\ell_{\delta,ij} = \log(W_\delta)_{ij}$ have row-softmax W_δ , induce the strip $B_h(t)$, and give

$$\Pi(\ell_\delta, h) = \text{diag}(s)W_\delta B_h(t)^\top.$$

For fixed δ , Lemma 1 sends this to $\text{diag}(s)W_\delta$ as $h \rightarrow 0$; sending $\delta \rightarrow 0$ then gives Π^* . A diagonal sequence $(\delta_k, h_k) \rightarrow (0, 0)$ therefore converges to Π^* . Conversely every output lies in the closed set $U(s, t)$ by Proposition 1, proving equality of the closure. In the square uniform case a permutation coupling is P/n , not P . $\square$

The completeness of Theorem 1 is a *closure* statement — boundary points are approached jointly through positive softmax weights and $h \rightarrow 0$. The relative interior is in fact reached *exactly*, at finite width, by an explicit closed-form inverse: *de-blurring* the target through the one strip.

Theorem 2 (Explicit exact inverse; onto the interior). *Let Π^* lie in the relative interior of $U(s, t)$, $\pi_{\min} := \min_{ij} \Pi_{ij}^* > 0$, and set $W(h) = \text{diag}(s)^{-1} \Pi^* B_h(t)^{-\top}$. There is a threshold $h_0 = h_0(\Pi^*) \in (0, h_1]$ — which must shrink as Π^* nears the boundary, at worst on the order $1/\log(1/\pi_{\min})$ up to constants depending on s, t and the grid — such that for every $h \leq h_0$: $W(h)$ is a positive, row-stochastic weight matrix with column mass t (so the strip $B_h(t)$ it induces is self-consistent), the map returns $\Pi(W(h), h) = \text{diag}(s) W(h) B_h(t)^\top = \Pi^*$ exactly, and $\|W(h) - \text{diag}(s)^{-1} \Pi^*\|_\infty = O(e^{-\rho/h})$. Consequently the family is onto the relative interior of $U(s, t)$: every strictly positive prescribed-marginal matrix is attained exactly, at all sufficiently small width — no iteration, no fixed point.*

Proof. Write $B := B_h(t)$, invertible for $h \leq h_1$ with $B^{-\top} = I + O(e^{-\rho/h})$ (Lemma 1). Column-stochasticity $\mathbf{1}^\top B = \mathbf{1}^\top$ gives $B^{-\top} \mathbf{1} = \mathbf{1}$, so $W(h) \mathbf{1} = \text{diag}(s)^{-1} \Pi^* \mathbf{1} = \text{diag}(s)^{-1} s = \mathbf{1}$: $W(h)$ is row-stochastic. Its column mass is $W(h)^\top s = B^{-1} \Pi^{*\top} \text{diag}(s)^{-1} s = B^{-1} \Pi^* \mathbf{1} = B^{-1} t = t$, using the marginal identity $Bt = t$; so the strip it induces is exactly the B that defines it, and $\Pi(W(h), h) = \text{diag}(s) W(h) B^\top = \Pi^* B^{-\top} B^\top = \Pi^*$. Expanding, $W(h) = \text{diag}(s)^{-1} \Pi^* (I + O(e^{-\rho/h})) = \text{diag}(s)^{-1} \Pi^* + O(e^{-\rho/h})$; writing the entrywise remainder as $\leq C' e^{-\rho/h}$ and setting $w_{\min} = \min_{ij} (\Pi_{ij}^* / s_i) > 0$, positivity follows whenever $C' e^{-\rho/h} \leq w_{\min}/2$. If $2C' > w_{\min}$, any $h_0 \leq \min\{h_1, \rho/\log(2C'/w_{\min})\}$ suffices; otherwise $h_0 = h_1$ already does. The guaranteed threshold therefore decreases logarithmically as the target approaches the boundary. As Π^* was arbitrary, the images cover the interior. $\square$

The width h controls smoothing along the ordered strip axis. It is *not* numerically equivalent to Sinkhorn’s entropic ε : the two parameters act on different maps and equal values need not produce plans of comparable entropy or sharpness. Within the present family, however, a prescribed interior Π^* can be reached at finite width by de-blurring its columns, $\text{diag}(s)^{-1} \Pi^* B_h(t)^{-\top}$, while boundary couplings require a joint positive-weight and sharp-width limit (Theorem 1). Appendix D provides the proof and a numerical check.

Differentiable scores, widths, and marginals. Because the map is closed-form, every ingredient is a gradient target. The strip B is differentiable in its inputs through the barrier map’s matrix-free implicit derivative (§3); the softmax W and the diagonal $\text{diag}(s)$ are elementary; the marginals s, t may be fixed inputs or learned — s enters through the diagonal and column mass $c = W^\top s$, while t determines the barrier levels T_j . Sinkhorn can also be differentiated with respect to its marginals and regularisation; the advantage here is that doing so does not require differentiating an iterative normalisation. Reverse mode runs back through the barrier map’s implicit VJP, the one dense product and the same prefix scan, so it never materialises the $\mathcal{O}((nm)^2)$ Jacobian of $\ell \mapsto \Pi$ — the property that makes the layer usable at scale. The *backward* is as parallel as the forward, the central advantage over both iterative normalisers and sequential parametrisations.

One logit head parametrises the layer. The width h is a free per-axis parameter, or its log-width can be reclaimed from the logits,

$$\log h = -\bar{\ell}, \quad h = \exp(-\bar{\ell}). \quad (11)$$

A global shift of ℓ leaves the row-wise softmax W unchanged, so its otherwise unused mean can control strip hardness without sacrificing any softmax degree of freedom. The sign matches the usual logit convention: increasing the mean decreases h and hardens the strip; decreasing it increases h and softens the strip. Working in $\log h$ keeps positivity explicit and avoids a numerical zero (the implementation clips

only at the finite-precision limits before exponentiation). Thus a *single* logit head parametrises (W, h) with no separate width output; the derivative in h is closed form (Appendix A). This reclamation does not reduce expressivity: for any positive row-stochastic W and desired h , choosing $\ell = \log W + c\mathbf{1}\mathbf{1}^\top$ with $c = -\log h - \text{mean}(\log W)$ leaves $\text{softmax}(\ell) = W$ and gives $-\bar{\ell} = \log h$. Only one modelling choice remains: which axis carries the strip. Placing it on the *shorter* side minimises the number of barriers, so the softmax is taken row-wise when $m \leq n$ and column-wise otherwise (the implementation just transposes); for a square map the row-wise softmax is also the memory-contiguous reduction. This is the whole deployment story in one line: `scores` $\mapsto$ exact prescribed-marginal matrix, one softmax and one strip.

4.1 The direct apply interface: scores and values in, transported values out

The public layer has two interfaces. `coupling` returns the matrix Π ; `apply` accepts the same scores together with values $X \in \mathbb{R}^{m \times d}$ and returns $Y = \Pi X$ directly. Here d is the number of value channels (attention features, transported features, or routed token features) sharing one coupling. The crucial point is that `apply` is *not* a matrix multiplication performed after constructing Π . The rows of B index the final coupling columns (and hence the rows of X), while its columns index the latent softmax columns. Thus $B^\top X$ pulls target-bin values back to the latent columns, W mixes those latent values into output rows, and $\text{diag}(s)$ assigns the required row mass. Associativity lets us evaluate these three factors of (10) from right to left:

$$\begin{aligned} Z &= B^\top X && \text{(strip the values),} \\ H &= WZ && \text{(mix with the row-softmax weights),} \\ Y &= \text{diag}(s)H && \text{(scale the output rows).} \end{aligned} \tag{12}$$

Although these operations are exactly equal to ΠX , neither the $m \times m$ strip B nor the $n \times m$ coupling Π is instantiated. The latent operator still has exact marginals by Proposition 1; `apply` merely evaluates its action more efficiently. The three stages are (Figure 3):

- (i) **Strip** $Z = B^\top X$ ($m \times d$). Each output row is a weighted average of the input rows across the column-quantile cuts. Since B is the strip (9), this is a single matrix-free *prefix scan* along the column axis — barriers and CDF differences applied on the fly — costing $\mathcal{O}(md)$ on the presorted grid, with B itself *never materialised*.
- (ii) **Mix** WZ ($n \times d$). The row-wise softmax combines those cut-values into the output rows: one dense $n \times m$ by $m \times d$ matrix product — the *single* $\mathcal{O}(nmd)$ step, implemented as a standard dense GEMM.
- (iii) **Scale** $\text{diag}(s)(WZ)$. Multiply row i by s_i — an elementwise $\mathcal{O}(nd)$ operation.

The heavy work after the one-pass barrier construction is one dense GEMM flanked by a scan and a diagonal. From raw scores, the complete path also includes the $\mathcal{O}(nm)$ softmax, column-mass, and barrier preprocessing; direct apply is therefore not a subquadratic claim. The dense $n \times m$ matrix W is still required, but neither B nor the additional $n \times m$ plan is stored. Beyond the scores and W , `apply` therefore uses $\mathcal{O}((n+m)d)$ working memory. It remains differentiable in ℓ, X, h, s, t , and its backward follows the same three stages in reverse.

This distinction also states the comparison to Sinkhorn precisely. After Sinkhorn has converged, one may evaluate $\text{diag}(u)K(\text{diag}(v)X)$ without materialising its final plan. What Sinkhorn does not provide is a *one-pass score-to-output* path: the scaling vectors u, v must first be produced to a requested feasibility tolerance by L alternating normalisation sweeps (and a converged derivative needs an unroll or a linear solve). LAPHORN’s `apply` instead maps (ℓ, X) to the action of an exact-marginal coupling in one fixed, non-iterative computation. Section 5 times the complete score-to-output paths, so the reported advantage is not a comparison against unnecessarily storing a Sinkhorn plan.

5 Numerical accuracy, derivative fidelity, and performance

The experiments answer four questions. First, does the implementation realise the mathematics to numerical precision, including its reverse pass? Second, what is gained by using the direct score-to-output apply rather than constructing Π ? Third, when are compiled PyTorch or fused CUDA preferable? Fourth, when does replacing iterative normalisation by a direct map matter relative to Sinkhorn? The

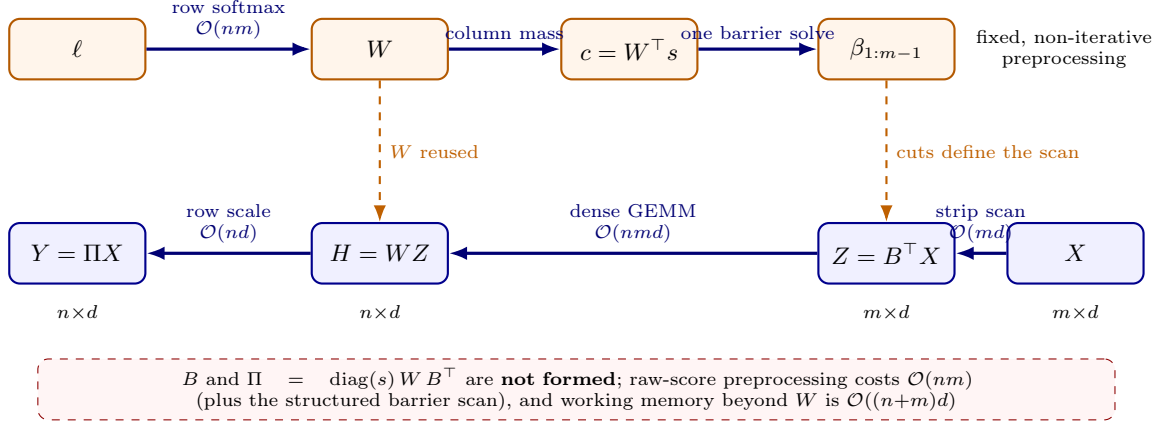


Figure 3: The complete direct score-to-output path. Scores produce the row-softmax weights, their column mass, and the $m - 1$ finite quantile cuts; the cuts define a matrix-free scan for $B^\top X$, after which one dense GEMM and one row scale return $Y = \text{diag}(s)W(B^\top X)$. Neither B nor Π is formed. Beyond W , working memory is $\mathcal{O}((n+m)d)$, and the backward traverses the same stages in reverse. Sinkhorn can likewise avoid its final plan, but only after iterative scaling has obtained u, v to the requested feasibility tolerance.

results answer these questions consistently. Both marginals are at the fp64 accumulation floor and the analytic VJP passes independent finite-difference checks; compiled PyTorch is already competitive with custom CUDA over much of the measured range; at $n = 4096$, direct apply is $5.1\times$ faster in forward and uses $3.0\times$ less incremental peak training memory than the best fresh-plan path; the best calibrated Sinkhorn+ X read-out requires $1.15\text{--}2.40\times$ its training peak; and Sinkhorn moves from faster on easy kernels to $6.9\times$ slower at $\varepsilon = 0.3$ for $n = 1024$, with the end-to-end multi-channel apply gap reaching $32.3\times$ at $n = 4096$. In the derivative audits, 61 and 108 Sinkhorn unrolls are required before truncation error falls below the predeclared finite-difference verification error measured for Laphorn’s scalar gradient and full Jacobian. The protocol controls feasibility and derivative error instead of assigning Sinkhorn an arbitrary fixed budget, and it never equates the unrelated smoothing parameters h and ε .

5.1 Deterministic implementation audit

Protocol. The audit script uses seed 20260718, a rectangular problem with $n = 6$, $m = 5$, feature width $d = 3$, random fp64 logits, and positive non-uniform marginals obtained by softmax. It is run both on CPU and on an RTX 5060 Laptop GPU with PyTorch 2.11.0. The checked objects are:

1. the structural full coupling, computed as $\text{diag}(s)(BW^\top)^\top$ by applying the strip directly to $W^\top$;
2. an explicit identity-based reference that forms $B^\top$ on the identity and then computes $WB^\top$;
3. the factorised action $\text{diag}(s)W(B^\top X)$;
4. both marginal residuals; and
5. the analytic VJP with respect to the logits.

The first two paths are compared only as a value check: the identity-based path is not used as a performance baseline. The full source of the audit is included with the implementation.

Machine-precision feasibility and an output-linear full-plan path. Both marginals agree with their targets at the fp64 accumulation floor (Table 1). The audit also closes the complexity gap between the mathematics and the code: forming the full coupling does not require materialising B on an identity matrix. Since the strip primitive computes BV directly, setting $V = W^\top$ gives

$$\Pi = \text{diag}(s)(BW^\top)^\top$$

in $\mathcal{O}(nm)$ work, followed only by the elementwise row scale. The result agrees with the explicit identity-based reference to below 10^{-16} , but avoids its unnecessary $\mathcal{O}(nm^2)$ dense multiplication. Thus exact feasibility is not only a theorem about an ideal factorisation: it is delivered by both public computational paths at machine precision.

maximum absolute error	CPU fp64	RTX 5060 fp64
structural plan vs. identity-based reference	8.3×10^{-17}	4.2×10^{-17}
factorised apply vs. ΠX	1.7×10^{-16}	1.4×10^{-16}
row marginal	5.6×10^{-17}	2.8×10^{-17}
column marginal	1.1×10^{-16}	2.9×10^{-16}
best relative directional-derivative error	1.0×10^{-10}	2.0×10^{-9}
full <code>gradcheck</code>	pass	pass

Table 1: Deterministic audit of the pure-PyTorch implementation ($n = 6$, $m = 5$, $d = 3$, $h = 0.35$, seed 20260718). The two devices evaluate the same fp64 protocol. These are implementation-consistency and derivative checks, not task-level accuracy measurements.

operation	leading work after the barrier solve	additional stored output/workspace
Laphorn direct $(\ell, X) \mapsto Y$	strip $\mathcal{O}(md)$ and GEMM $\mathcal{O}(nmd)$	$\mathcal{O}((n+m)d)$ beyond W
Laphorn full $\ell \mapsto \Pi$	direct strip $BW^\top$: $\mathcal{O}(nm)$	$\mathcal{O}(nm)$ for the requested plan
full Π , identity diagnostic	form B and multiply $WB^\top$: $\mathcal{O}(m^2 + nm^2)$	$\mathcal{O}(m^2 + nm)$; reference only
Sinkhorn operator after scaling	GEMM $\mathcal{O}(nmd)$; obtaining u, v costs L kernel sweeps	kernel plus u, v ; no extra plan required

Table 2: The operations relevant to deployment. Both methods can avoid storing a final dense plan, but at different points in the computation: Laphorn’s direct apply is a one-pass map from scores and values to Y , whereas the Sinkhorn operator is available only after L scaling sweeps have produced u, v . The identity-based Laphorn route is a diagnostic, not a performance baseline.

5.2 Validation of the analytic reverse pass

We distinguish the score matrix from the upstream cotangent. Let ℓ denote the logits, $G \in \mathbb{R}^{n \times m}$ a fixed random matrix, and

$$L(\ell) = \langle \Pi(\ell), G \rangle.$$

For a unit direction v we compare the analytic contraction $\langle \nabla_\ell L, v \rangle$ with the central difference

$$D_a(v) = \frac{L(\ell + av) - L(\ell - av)}{2a}, \quad a \in \{10^{-3}, 3 \times 10^{-4}, 10^{-4}, 3 \times 10^{-5}, 10^{-5}, 3 \times 10^{-6}\}.$$

The best relative discrepancy over this step sweep is 1.0×10^{-10} on CPU and 2.0×10^{-9} on the RTX 5060 in fp64. In addition, PyTorch’s elementwise central-difference `gradcheck` passes for the complete local Jacobian of $\ell \mapsto \Pi$. These tests numerically validate the analytic, scan-based VJP; we therefore use “analytic VJP” rather than calling a finite-difference number itself an exact gradient.

The practical point is stronger than passing a derivative test. Laphorn has no iteration axis in its backward map: once the one-pass forward is specified, its VJP is a fixed collection of scans and tensor contractions. Section 5.5 measures how many iterations a convergent normaliser needs to attain the same numerical derivative fidelity.

This audit does not claim that a particular truncated Sinkhorn derivative is “incorrect”: it is the exact derivative of the truncated L -step map. A comparison to the derivative of the *converged* Sinkhorn map must jointly control the forward residual and the adjoint error, and must report the stopping criterion rather than assume a universal iteration count.

5.3 Two deployment interfaces: full coupling and direct apply

Laphorn exposes two useful operations. The full-coupling interface maps scores to Π when the matrix itself is an output or will be reused. The direct-apply interface maps (ℓ, X) straight to $Y = \Pi X$ when only transported values are needed. It evaluates $B^\top X$ first, so neither the strip nor the coupling is constructed. A third, identity-based route is retained only as a diagnostic. Table 2 separates these operations. Direct apply saves the additional plan allocation and the work of producing it; it does not remove the dense score/softmax matrix W or make dense attention subquadratic.

$n = m$	eager apply	compiled apply	CUDA apply	eager plan	compiled plan	CUDA plan
256	2.89 / 8.75	0.73 / 3.60	0.66 / 2.95	2.95 / 8.61	0.71 / 3.40	0.73 / 3.08
512	2.81 / 8.01	0.72 / 2.93	0.65 / 2.68	2.82 / 8.01	0.69 / 3.08	0.71 / 2.75
1024	2.97 / 7.63	0.75 / 2.84	0.67 / 2.49	3.09 / 10.39	0.73 / 2.35	0.74 / 2.67
2048	2.59 / 6.96	0.66 / 2.67	0.60 / 2.53	7.70 / 39.74	1.74 / 7.94	1.16 / 4.57
4096	4.08 / 14.19	1.18 / 4.40	1.35 / 6.67	30.29 / 175.14	9.04 / 42.42	6.00 / 23.98

Table 3: GPU time in ms for forward / forward+backward at $B = 1$ and $d = 64$. The plan columns include constructing a fresh Π and applying it once; they do not measure reuse of a prebuilt plan. Bold marks the lowest median in each row and phase. The $n = 4096$ row was rerun after the peak-memory audit established that all six single-step paths fit on the device.

sweep	value	eager apply	compiled apply	CUDA apply	eager plan	compiled plan	CUDA plan
$d (B = 1)$	16	7.89	2.99	2.55	10.38	2.53	2.86
	64	7.63	2.84	2.49	10.39	2.35	2.67
	256	7.63	3.04	2.52	9.69	2.51	2.66
$B (d = 64)$	1	7.63	2.84	2.49	10.39	2.35	2.67
	2	14.95	5.35	4.94	20.70	4.68	5.16
	4	28.40	9.13	9.31	41.56	8.27	9.07
	8	56.51	17.72	18.05	83.80	17.06	17.84

Table 4: Median forward+backward time in ms at $n = m = 1024$. The public implementation is per instance, so a batch is B independent calls accumulated into one loss; no method receives an unpublished native-batch kernel. Consequently the batch sweep measures latency/throughput scaling of the released interface, not the ceiling of a future fused batched implementation.

5.4 Implementation time and training memory

We next time six computational paths, without introducing a Sinkhorn comparison. The platform is a laptop RTX 5060 (8 GB), PyTorch 2.11.0+cu128 and CUDA 12.8, in fp32. Scores are square, standard-normal matrices, marginals are uniform, and the default trainable width is $\log h = -\text{mean}(\ell)$. “Apply” means the complete direct path $(\ell, X) \mapsto Y$ without forming B or Π ; “plan” builds the structural full coupling and immediately multiplies it by X . Each compiled path uses a fixed-shape `torch.compile` graph, with `fullgraph=False` and `dynamic=False`, separately for the matrix-free operator and the full-plan construction, with distinct graphs for inference and training. First-call compilation and CUDA-extension JIT are excluded.

We warm the GPU, rotate method order between samples, and use CUDA events. A forward sample is the per-call average of 10 invocations (30 samples); a forward+backward sample averages 5 invocations (20 samples). Tables report medians. The median relative IQR over all 119 successful cells is 8.8% and its 90th percentile is 14.2%; the accompanying benchmark script reports every Q1 and Q3. Before timing, every path is checked against eager apply; the largest fp32 discrepancy over the sweep is 4.9×10^{-7} .

Speed is only half of the deployment cost. We therefore measure the incremental peak allocated GPU memory of one warmed-up forward+backward step for the same six paths. Immediately before each step we synchronise, release unreferenced tensors and allocator cache, record `memory_allocated`, and reset PyTorch’s peak statistics. We report `max_memory_allocated` minus that baseline, not reserved cache. The metric excludes the already-live inputs, probe, loaded extensions, and persistent compiled state, but includes forward temporaries, tensors saved for backward, the output, and both returned gradients. Three repeats agree at the displayed precision. This definition isolates the additional memory needed to train with a warmed implementation rather than folding one-time compilation into the layer.

sweep	value	eager apply	compiled apply	CUDA apply	eager plan	compiled plan	CUDA plan
n ($B = 1$)	256	3.75	2.13	1.13	13.14	6.64	2.39
	512	8.51	5.27	4.25	52.27	27.02	9.28
	1024	21.02	14.53	16.51	208.54	104.55	36.57
	2048	65.01	49.04	65.01	833.07	417.10	145.12
	4096	258.02	194.07	258.02	3330.14	1666.19	578.24
B ($n = 1024$)	2	31.07	23.82	25.26	297.59	177.35	45.59
	4	58.37	56.07	48.04	475.19	322.44	71.11
	8	129.80	125.21	101.11	830.40	612.64	122.18

Table 5: Incremental peak allocated GPU memory in MiB for one fp32 forward+backward step at $d = 64$. Each plan path constructs a fresh coupling and applies it once. The $B = 1, n = 1024$ base case appears in the size sweep. Bold marks the smallest apply footprint in each row; every apply path is also smaller than its plan-building counterpart.

Three conclusions are visible. First, feature width has little effect over $d = 16$ –256 because the dense score/softmax and barrier work dominate this regime. Second, batching the released per-instance interface is close to linear. At $n = 1024$, compiling the complete plan construction is the fastest reverse path throughout the d and B sweeps; compilation therefore removes the need for custom CUDA in this regime. Third, the conclusion changes with scale. At $n = 2048$ and 4096, CUDA full-plan construction is about $1.5\times$ faster in forward and 1.7 – $1.8\times$ faster in forward+backward than compiled full-plan construction. When only the action ΠX is required, factorisation matters more than kernel choice: at $n = 4096$ the best apply path is $5.1\times$ faster in forward and $5.4\times$ faster in forward+backward than the best path that constructs a fresh plan. It also needs 194 versus 578 MiB of incremental peak training memory, a $3.0\times$ reduction. Within the same compiled-PyTorch backend the reduction is $8.6\times$ (194 versus 1666 MiB), and within eager PyTorch it is $12.9\times$. Direct apply still has quadratic storage through the dense scores, softmax weights, and their gradients; the measured saving comes from eliminating the fresh coupling and the plan-construction intermediates that autograd would otherwise retain. In the batch sweep the gap narrows because the released interface executes independent instances and common output/gradient accumulation becomes more visible, but apply remains smaller for every paired implementation and batch. Materialising a coupling is therefore appropriate mainly when the plan itself is an output or will be reused; otherwise direct apply is the clear deployment path. This is the systems contribution of the factorisation: the layer offers the action of an exact-marginal coupling as a first-class operation, rather than forcing every caller to construct the coupling and multiply it afterward. These implementation timings do not select a transport objective and therefore are not an accuracy-matched comparison to Sinkhorn.

5.5 Feasibility- and derivative-controlled Sinkhorn comparison

We instantiate the protocol rather than compare against a fixed iteration budget. The experiment is *feasibility-matched*, not plan- or objective-matched: Laphorn and entropic Sinkhorn define different maps, and $h = \varepsilon$ is not a matched-sharpness condition. Scores are standard normal, $s = t = \mathbf{1}/n$, $B = 1$, $d = 64$, and all timed paths are fp32 on the same RTX 5060. For Sinkhorn we explicitly construct the globally shifted dense log-kernel $\log K = \ell/\varepsilon - \max(\ell/\varepsilon)$, update both diagonal factors in the log domain, and apply $\text{diag}(u)K \text{diag}(v)X$ without forming an additional Π . The stopping error is the largest absolute row or column discrepancy divided by its uniform target mass. We determine the first successful full iteration outside the timed region and then time that fixed solve. Loops of at most 16 iterations are compiled end to end; longer solves reuse a compiled eight-iteration block, avoiding an impractically large unrolled graph. First compilation is excluded. Each entry is the median of 20 CUDA-event samples after five warm-ups; the median relative IQR is 6.8% and its 90th percentile is 11.8%.

n	Laphorn, exact marginals		Sinkhorn eager	Sinkhorn compiled	
	compiled	CUDA	$\tau = 10^{-6}$	$\tau = 10^{-4}$	$\tau = 10^{-6}$
256	0.60	0.60	1.26 (4)	0.38 (3)	0.44 (4)
512	0.61	0.58	1.43 (4)	0.38 (3)	0.48 (4)
1024	0.57	0.53	1.03 (3)	0.34 (2)	0.37 (3)
2048	0.58	0.64	1.51 (3)	0.38 (2)	0.46 (3)

Table 6: Forward time in ms for constructing the factors and applying them to X . Parentheses give the number of complete Sinkhorn row/column iterations required to reach relative marginal residual τ . Laphorn uses explicit $h = 1$ and is feasible by construction.

The comparison includes Sinkhorn’s favourable regime. For well-conditioned $\varepsilon = 1$ scores, compiled Sinkhorn reaches 10^{-6} in only three or four iterations and is faster than either Laphorn path (Table 6). This is a genuine win for Sinkhorn, and it identifies the crossover rather than weakening the case for a direct map: as the kernel sharpens, the fixed cost of Laphorn stays nearly unchanged while Sinkhorn’s iteration count grows. Table 7 separates entropic scale from score conditioning at $n = 1024$; the target is 10^{-6} throughout.

sweep	value	iterations	achieved residual	compiled time (ms)
ε (score std = 1)	0.1	512	1.75×10^{-3}	31.06
	0.3	55	9.54×10^{-7}	3.93
	1.0	3	9.54×10^{-7}	0.41
	3.0	2	9.54×10^{-7}	0.31
score std ($\varepsilon = 1$)	0.5	2	9.54×10^{-7}	0.32
	1.0	3	9.54×10^{-7}	0.37
	2.0	12	5.36×10^{-7}	1.04
	4.0	106	9.54×10^{-7}	7.23

Table 7: Sinkhorn sensitivity to entropic scale and score conditioning at $n = 1024$, $d = 64$. The $\varepsilon = 0.1$ run is explicitly reported as a capped failure: it does not reach the target within 512 iterations.

The independent Laphorn width sweep $h \in \{0.1, 0.3, 1, 3, 10\}$ remains between 0.59–0.64 ms for compiled PyTorch and 0.54–0.58 ms for CUDA, with exact marginals in every case. Thus the Laphorn cost is insensitive to its width in this regime, whereas Sinkhorn’s time-to-feasibility varies from 0.31 ms to more than 31 ms and may miss the requested residual. At the directly comparable standard-score setting $\varepsilon = 0.3$, compiled Sinkhorn needs 55 iterations and 3.93 ms versus 0.57 ms for compiled Laphorn at $n = 1024$: a $6.9\times$ gap while Laphorn satisfies both marginals by construction. At $\varepsilon = 0.1$, Sinkhorn still has residual 1.75×10^{-3} after 512 iterations; Laphorn has no analogous convergence failure mode because feasibility is algebraic rather than asymptotic.

The end-to-end score-to-output comparison. The preceding comparison fixes $d = 64$ and uses the plan-free factorised Sinkhorn read-out. We now test the deployment interface directly. A single score matrix acts on $X \in \mathbb{R}^{n \times d}$, so d is the number of value channels processed by the same coupling. Independent standard-normal score draws use fixed seeds 20260740 and 20260741 for $n = 1024$ and 4096, respectively. We time the complete path from scores and values to Y . Compiled Laphorn uses its direct apply without constructing B or Π ; the conventional Sinkhorn path computes its calibrated plan and then executes one dense ΠX multiply. Iteration calibration, compilation, and warm-up remain outside the timed region. As a control we also timed the structurally plan-free factorised Sinkhorn operator $\text{diag}(u)K \text{diag}(v)X$. Materialising Π was never more than 4% slower and was often faster on this GPU, so the comparison below is not created by an inefficient post-processing implementation. The two Sinkhorn read-outs agree to at most 2.5×10^{-6} in fp32. Thus Table 8 measures the distinction that matters: Laphorn evaluates an exactly feasible action in one pass, whereas Sinkhorn must first complete its normalisation loop even if its final plan is never stored.

n	d	Laphorn		$\varepsilon = 1$ Sinkhorn		$\varepsilon = 0.3$ Sinkhorn	
		compiled	apply	plan+ $\times$	speed-up	plan+ $\times$	speed-up
1024	8		0.68	0.34	$0.50\times$	4.84	$7.08\times$
1024	64		0.75	0.35	$0.46\times$	4.96	$6.64\times$
1024	256		0.64	0.38	$0.58\times$	5.04	$7.83\times$
1024	1024		0.83	0.46	$0.55\times$	5.20	$6.29\times$
4096	8		1.20	2.20	$1.84\times$	33.13	$27.71\times$
4096	64		1.03	2.41	$2.35\times$	33.22	$32.33\times$
4096	256		1.62	2.51	$1.54\times$	33.75	$20.80\times$
4096	1024		4.61	3.91	$0.85\times$	35.16	$7.62\times$

Table 8: Median forward time in ms for the complete multi-channel application $\ell, X \mapsto Y$ on the RTX 5060. Sinkhorn is calibrated to relative marginal residual 10^{-6} , requiring three iterations at $\varepsilon = 1$ and 66/68 iterations at $\varepsilon = 0.3$ for $n = 1024/4096$. Laphorn uses $h = 1$ and has exact marginals by construction. “Speed-up” is Sinkhorn time divided by Laphorn time; values below one identify a Sinkhorn win. Each entry is the median of 20 CUDA-event samples after five warm-ups.

The value-width sweep sharpens the crossover. At $n = 1024$, the three-iteration $\varepsilon = 1$ solve remains genuinely faster, by 1.7–2.2 $\times$. At $n = 4096$, however, Laphorn already wins by 1.5–2.4 $\times$ for $d \leq 256$ even in this easy regime; only the very wide $d = 1024$ multiply lets Sinkhorn recover a modest 1.18 $\times$ advantage. In the sharper $\varepsilon = 0.3$ regime, the non-iterative path is 6.3–7.8 $\times$ faster at $n = 1024$ and 7.6–32.3 $\times$ faster at $n = 4096$. At $n = 4096, d = 64$, compiled Laphorn takes 1.03 ms against 33.22 ms for 68 Sinkhorn iterations plus the multi-channel multiply. Increasing d eventually makes the dense read-out shared by both methods dominate, which explains the narrowing at $d = 1024$ rather than hiding it.

Peak memory for Sinkhorn followed by X . We now apply the incremental peak-allocation protocol of Table 5 to the same score-to-output comparison at $d = 64$. In addition to Laphorn’s compiled direct apply, we measure both Sinkhorn read-outs: the plan-free $\text{diag}(u)K\text{diag}(v)X$ operator and the conventional path that materialises the calibrated Π and then computes ΠX . Calibration, compilation, and warm-up precede the measurement. Inputs, the probe, and persistent compiler state are excluded; forward temporaries, tensors saved for backward, output, and returned gradients are included. All three repeated peaks agree at the displayed precision.

n	ε (iterations)	forward			forward+backward		
		Laphorn apply	Sinkhorn factorised	Sinkhorn plan+ X	Laphorn apply	Sinkhorn factorised	Sinkhorn plan+ X
1024	1.0 (3)	5.77	8.04	4.25	15.52	25.77	25.52
1024	0.3 (66)	5.77	8.51	8.01	15.52	17.85	21.09
4096	1.0 (3)	71.06	128.06	65.00	194.06	467.06	466.06
4096	0.3 (68)	71.06	130.03	128.03	194.06	274.00	324.42

Table 9: Incremental peak allocated GPU memory in MiB for the complete compiled $\ell, X \mapsto Y$ path at $d = 64$. Sinkhorn is calibrated to relative marginal residual 10^{-6} ; Laphorn uses $h = 1$ and is feasible by construction. Bold marks the smallest footprint in each row and phase.

The forward-only result retains Sinkhorn’s favourable regime: with only three iterations, its materialised plan path uses slightly less incremental memory than Laphorn (4.25 versus 5.77 MiB at $n = 1024$, and 65.00 versus 71.06 MiB at $n = 4096$). Training reverses the comparison. Laphorn’s forward+backward peak is lower in every row and is 1.15–2.40 $\times$ smaller than the better of the two Sinkhorn read-outs. At $n = 4096$, the peaks are 194 versus 466 MiB for the easy kernel and 194 versus 274 MiB for the sharper kernel. The latter gap is not proportional to 68 iterations because the released implementation reuses a compiled eight-iteration block rather than unrolling one enormous graph; Table 9 therefore compares practical deployments rather than an intentionally memory-heavy baseline. Its backward differentiates the forward-residual-calibrated truncated Sinkhorn map. It is not a derivative-accuracy-matched

claim about the converged map; that stricter distinction is tested next.

We first differentiate a fixed linear probe at $n = 128, d = 16$. The Sinkhorn reference is an fp64 map run substantially past a 10^{-8} forward residual; we compare the gradient of the truncated map at its 10^{-6} forward stopping point. Table 10 reports relative ℓ_2 errors against that reference. This is a useful engineering tolerance and runtime comparison, but its 10^{-4} derivative criterion is much looser than the numerical accuracy of Laphorn’s analytic derivative.

ε	timed / ref. iter.	logit grad. err.	value grad. err.	Sinkhorn	Laphorn compiled	Laphorn CUDA
1.0	4 / 26	8.89×10^{-7}	2.08×10^{-7}	1.00	2.11	1.97
0.3	28 / 76	1.43×10^{-6}	1.30×10^{-7}	4.20	2.26	1.94

Table 10: Forward+backward time in ms and truncated-Sinkhorn derivative error. Laphorn’s analytic VJP differentiates a different map, so its columns are runtime comparisons, not errors against the Sinkhorn derivative.

Forward feasibility is not backward fidelity. We therefore repeat the derivative experiment with a predeclared numerical threshold. Laphorn’s finite-difference discrepancy is a *verification resolution*, not an intrinsic error estimate for its analytic derivative. We ask how many Sinkhorn iterations are needed before truncation error relative to the converged Sinkhorn derivative falls below that measured resolution. We fix the protocol before evaluating either method: $\varepsilon = h = 0.05$, nonuniform positive random marginals, and scores $\ell = -U$ with $U_{ij} \sim \mathcal{U}[0, 1]$, all in fp64. Laphorn’s target is not assumed: it is the smallest relative error over a predeclared central-difference step grid. For Sinkhorn, the reference derivative is obtained by differentiating a fixed point converged to relative marginal residual below 10^{-13} and solving its gauge-fixed implicit system; an independent directional finite-difference check gives errors 6.7×10^{-11} and 1.3×10^{-10} in the two tests. Thus the reference is more accurate than the Laphorn verification threshold. The two Sinkhorn columns below are the derivative of the K -step unrolled map and a K -step Neumann solve of the converged fixed-point derivative, respectively. The latter is an *additional* linear solve after the forward fixed point, not a replacement for its forward iterations.

K	forward plan error	unrolled gradient error	implicit-solve error
1	4.17×10^{-1}	8.45×10^{-1}	5.08×10^{-1}
3	1.59×10^{-1}	3.36×10^{-1}	1.63×10^{-1}
10	8.53×10^{-3}	2.37×10^{-2}	7.14×10^{-3}
30	5.87×10^{-6}	3.52×10^{-5}	5.15×10^{-6}
54	1.16×10^{-9}	1.21×10^{-8}	1.02×10^{-9}
61	9.64×10^{-11}	1.13×10^{-9}	8.51×10^{-11}
100	3.89×10^{-14}	5.96×10^{-14}	7.15×10^{-16}
Laphorn (one pass)		1.29×10^{-9} verification threshold	

Table 11: Derivative of $\langle \Pi, C \rangle$ with respect to the scores at $n = 20$. Errors are relative ℓ_2 errors against each method’s own numerical reference. The first Sinkhorn unrolling whose truncation error falls below Laphorn’s finite-difference verification threshold is $K = 61$; the implicit route needs a converged forward solve (83 iterations here) followed by 54 linear-solve steps.

The scalar loss can hide poorly converged directions, so we also repeat the comparison for the complete Jacobian $\partial \Pi / \partial \ell$ at $n = 8$. Laphorn’s analytic Jacobian has relative Frobenius discrepancy 8.18×10^{-10} against central differences in one pass; we use this as the verification threshold. Sinkhorn truncation falls below it only after 108 unrolled iterations. The implicit route needs a forward fixed point (139 iterations to the 10^{-13} residual here) and 96 iterations of its derivative solve.

K	unrolled Jacobian error	implicit-solve error
1	7.01×10^{-1}	5.47×10^{-1}
3	3.28×10^{-1}	2.11×10^{-1}
10	1.75×10^{-1}	3.45×10^{-2}
30	1.93×10^{-3}	5.48×10^{-4}
96	8.04×10^{-9}	6.87×10^{-10}
107	9.42×10^{-10}	7.13×10^{-11}
108	7.75×10^{-10}	5.80×10^{-11}
Laphorn	8.18×10^{-10} verification threshold (one pass)	

Table 12: Relative Frobenius error of the full $n^2 \times n^2$ plan Jacobian at $n = 8$. Forward feasibility alone does not certify this derivative: 108 Sinkhorn unrolls are required before truncation error falls below Laphorn’s measured finite-difference verification threshold.

The comparison gives a more useful conclusion than “one method is always faster.” Compiled Sinkhorn is preferable for easy, high-entropy kernels that converge in a handful of iterations. Laphorn removes the convergence axis: its cost and exact feasibility persist as the analogous Sinkhorn kernel becomes sharp or poorly conditioned. More decisively for differentiation, a marginal-residual stopping rule does not certify the backward map. In this controlled instance, Sinkhorn truncation needs 61 iterations for the scalar gradient and 108 for the full Jacobian to fall below Laphorn’s predeclared finite-difference verification thresholds; the implicit alternative adds 54 or 96 derivative solve steps after a converged forward solve. These are numerical-layer comparisons, not claims that the two maps optimise the same objective, but they isolate precisely the iterative work that Laphorn eliminates.

5.6 Scope of the empirical evidence

The evidence above validates the mathematical and implementation-level claims studied here: exact feasibility, full interior expressivity, a numerically checked analytic derivative, and practical PyTorch/CUDA execution. It deliberately does not claim that replacing an objective-selected Sinkhorn plan improves every trained model. That question is application-dependent and should be tested in doubly-stochastic attention, balanced routing, matching, or mHC at matched task quality, including optimisation stability, end-to-end throughput, memory, and sensitivity to the strip width and coordinate order. Likewise, exact Birkhoff charts overlap only in the square-uniform case and should be compared using matched execution paths. These boundaries are substantive: the paper’s claim is a stronger coupling *layer primitive*, not a universal replacement for optimal transport or every intrinsic polytope chart.

6 Discussion

The main result is not merely that LAPHORN is another parametrisation of a transportation polytope. It changes the computational route to feasibility: scores are read directly into a coupling, so exact marginals and the analytic backward arrive after a fixed, non-iterative computation rather than after convergence. This distinction is most valuable when the prescribed-marginal constraint is the layer interface. It also yields a second interface that is easy to miss if one thinks only in terms of matrices: when a model needs $Y = \Pi X$, `apply` maps scores and values directly to Y . It does not first construct Π , and it does not first run an iterative normaliser. This is stronger than merely applying an already converged factorised plan.

The distinction explains both empirical regimes. A three-step compiled Sinkhorn solve can be faster on an easy kernel, but Laphorn’s cost does not grow when sharpening pushes Sinkhorn to 55, 106, or more than 512 iterations. The same point explains the backward result: there is no gap between a stopping rule for the forward and the accuracy of a separately converged derivative, because Laphorn has no truncated fixed point.

The systems results sharpen the same message. The layer does not depend on a bespoke kernel to be practical: `torch.compile` matches or beats fused CUDA in several central regimes, while the CUDA path remains useful for large full-plan construction. More importantly, choosing the right mathematical interface dominates low-level tuning. At $n = 4096$, direct apply is over $5\times$ faster than constructing a fresh Laphorn plan and then using it. The user can therefore request either the coupling or its action without changing the represented map. The direct Sinkhorn comparison makes the deployment consequence concrete: for a calibrated $\varepsilon = 0.3$ solve followed by a multi-channel multiply, compiled Laphorn is 7.6–32.3 $\times$ faster at $n = 4096$. The advantage narrows as the value width grows because the final dense

multiply is shared, while at $\varepsilon = 1$ the measured crossover remains visible rather than being averaged away.

These advantages do not turn Laphorn into an optimal-transport solver. When the goal is the minimiser of a specified regularised OT objective, Sinkhorn’s objective-selected coupling is the desired output. When the goal is a free-standing square-uniform Birkhoff coordinate, intrinsic charts can use fewer parameters. Laphorn targets the complementary and common neural-layer case: an arbitrary score matrix must become a differentiable feasible coupling, possibly rectangular and with non-uniform marginals.

The directional construction has three limitations. First, the barrier axis is ordered. Sinkhorn is permutation-equivariant, whereas permuting the coordinates before applying LAPHORN changes adjacency and can change the returned coupling. Tokens, ranks, and spatial positions have a natural order; unordered experts or set elements require an explicit or learned ordering, and sensitivity to that choice should be validated in the downstream task. Second, exact sparse boundary couplings are obtained only in the limit $h \rightarrow 0$. The finite-width map covers the relative interior, but conditioning deteriorates near the boundary. Third, the factorised apply does not remove the dense score/softmax matrix W . It avoids the additional strip and coupling materialisations; it does not turn dense pairwise attention into a subquadratic method.

The experiments validate the mathematical and implementation-level claims studied here: feasibility, expressivity, implementation agreement, derivative verification, and systems cost. What remains application-dependent is whether the different inductive bias improves task quality. An end-to-end study in attention, balanced routing, or matching should therefore measure optimisation stability, sensitivity to coordinate order and width, and throughput at matched quality; it should not be conflated with the numerical question this paper resolves.

7 Conclusion

We introduced LAPHORN, which turns arbitrary scores into a prescribed-marginal coupling with one softmax, one quantile-barrier solve, and one strip. The construction is exact for every positive width, onto the relative interior at finite width, and dense in the full transportation polytope. Its backward is an explicit scan-based VJP. The layer also exposes two first-class interfaces: a direct $\mathcal{O}(nm)$ construction when the coupling itself is needed, and a one-pass $(\ell, X) \mapsto Y$ apply when only its action is needed. The latter evaluates $B^\top X$, then $W(\cdot)$, then the row scale. Sinkhorn can avoid storing its final plan, but still needs the preceding scaling iterations that LAPHORN removes.

The measurements make these structural claims operational. Marginals and path agreement reach the fp64 floor, and the analytic derivative passes directional differences and full `gradcheck`. Compiled Sinkhorn remains the right fast baseline for easy kernels, but at $n = 1024$ and $\varepsilon = 0.3$ Laphorn is $6.9\times$ faster while enforcing exact rather than tolerance-level feasibility; a sharper $\varepsilon = 0.1$ Sinkhorn run misses the target after 512 iterations. For the complete score-to-multi-channel-output path, the same advantage reaches $32.3\times$ at $n = 4096, d = 64$ (1.03 versus 33.22 ms), with a control showing that a factorised Sinkhorn read-out does not remove the gap. In the derivative audit, Sinkhorn requires 61 and 108 unrolled iterations before truncation error falls below the predeclared finite-difference verification error measured for Laphorn’s scalar gradient and full Jacobian, respectively. The central contribution is therefore a new layer primitive, not a faster approximation to the same OT objective: when a network needs a score-conditioned feasible coupling or its action, Laphorn removes iterative normalisation from the forward, apply, and derivative.

References

- [1] A. Grover, E. Wang, A. Zweig, and S. Ermon. Stochastic Optimization of Sorting Networks via Continuous Relaxations. *ICLR*, 2019. [arXiv:1903.08850](#).
- [2] M. Blondel, O. Teboul, Q. Berthet, and J. Djolonga. Fast Differentiable Sorting and Ranking. *ICML*, 2020. [arXiv:2002.08871](#).
- [3] S. Prillo and J. M. Eisenschlos. SoftSort: A Continuous Relaxation for the argsort Operator. *ICML*, 2020. [arXiv:2006.16038](#).

- [4] F. Petersen, C. Borgelt, H. Kuehne, and O. Deussen. Differentiable Sorting Networks for Scalable Sorting and Ranking Supervision. *ICML*, 2021. [arXiv:2105.04019](#).
- [5] Ł. Struski, M. B. Bednarczyk, I. T. Podolak, and J. Tabor. LapSum – One Method to Differentiate Them All: Ranking, Sorting and Top- k Selection. *ICML*, 2025. [arXiv:2503.06242](#).
- [6] M. Cuturi, O. Teboul, and J.-P. Vert. Differentiable Ranking and Sorting using Optimal Transport. *NeurIPS*, 2019. [arXiv:1905.11885](#).
- [7] F. Petersen, C. Borgelt, H. Kuehne, and O. Deussen. Monotonic Differentiable Sorting Networks. *ICLR*, 2022. [arXiv:2203.09630](#).
- [8] M. Cuturi. Sinkhorn Distances: Lightspeed Computation of Optimal Transport. *NeurIPS*, 2013. [arXiv:1306.0895](#).
- [9] R. Sinkhorn. A Relationship Between Arbitrary Positive Matrices and Doubly Stochastic Matrices. *Ann. Math. Statist.*, 35(2):876–879, 1964.
- [10] R. Sinkhorn and P. Knopp. Concerning Nonnegative Matrices and Doubly Stochastic Matrices. *Pacific J. Math.*, 21(2):343–348, 1967.
- [11] G. Birkhoff. Three Observations on Linear Algebra. *Univ. Nac. Tucumán, Rev. Ser. A*, 5:147–151, 1946.
- [12] S. W. Linderman, G. E. Mena, H. Cooper, L. Paninski, and J. P. Cunningham. Reparameterizing the Birkhoff Polytope for Variational Permutation Inference. *AISTATS*, 2018. [arXiv:1710.09508](#).
- [13] A. Lyubinin. TBP-mHC: Full Expressivity for Manifold-Constrained Hyper Connections through Transportation Polytopes. 2026. [arXiv:2605.21724](#).
- [14] C. Wang and Y. Qiu. Accelerating Birkhoff Projection for Manifold-Constrained Hyper-Connections. 2026. [arXiv:2606.07574](#).
- [15] R. P. Adams and R. S. Zemel. Ranking via Sinkhorn Propagation. 2011. [arXiv:1106.1925](#).
- [16] G. Peyré and M. Cuturi. Computational Optimal Transport. *Foundations and Trends in Machine Learning*, 11(5–6):355–607, 2019. [arXiv:1803.00567](#).
- [17] J. Feydy, T. Séjourné, F.-X. Vialard, S.-i. Amari, A. Trounev, and G. Peyré. Interpolating between Optimal Transport and MMD using Sinkhorn Divergences. *AISTATS*, 2019. [arXiv:1810.08278](#).
- [18] R. Flamary, N. Courty, A. Gramfort, et al. POT: Python Optimal Transport. *JMLR*, 22(78):1–8, 2021. [JMLR](#).
- [19] N. Bonneel, J. Rabin, G. Peyré, and H. Pfister. Sliced and Radon Wasserstein Barycenters of Measures. *Journal of Mathematical Imaging and Vision*, 51(1):22–45, 2015.
- [20] G. Mena, D. Belanger, S. Linderman, and J. Snoek. Learning Latent Permutations with Gumbel-Sinkhorn Networks. *ICLR*, 2018. [arXiv:1802.08665](#).
- [21] A. Myronenko and X. Song. Point Set Registration: Coherent Point Drift. *IEEE TPAMI*, 32(12):2262–2275, 2010. [arXiv:0905.2635](#).
- [22] P.-E. Sarlin, D. DeTone, T. Malisiewicz, and A. Rabinovich. SuperGlue: Learning Feature Matching with Graph Neural Networks. *CVPR*, 2020. [arXiv:1911.11763](#).
- [23] Y. Zhu, Z. Zhang, Y. Wang, et al. Differentiable Fast Top-K Selection for Large-Scale Recommendation. 2025. [arXiv:2510.11472](#).
- [24] Z. Xie, Y. Wei, H. Cao, C. Zhao, D. Dai, H. Gao, et al. mHC: Manifold-Constrained Hyper-Connections. 2025. [arXiv:2512.24880](#).
- [25] J. Born, F. Skogh, K. Rhrissorrakrai, F. Utro, N. Wagner, and A. Sobczyk. Quantum Doubly Stochastic Transformers. *NeurIPS*, 2025. [arXiv:2504.16275](#).

- [26] M. E. Sander, P. Ablin, M. Blondel, and G. Peyré. Sinkformers: Transformers with Doubly Stochastic Attention. *AISTATS*, 2022. [arXiv:2110.11773](#).
- [27] D. A. Nguyen, H. B. Ta, N. Le Duc, T. M. Nguyen, and T. Tran. Selective Sinkhorn Routing for Improved Sparse Mixture of Experts. 2025. [arXiv:2511.08972](#).
- [28] A. Clark, D. de las Casas, A. Guy, A. Mensch, et al. Unified Scaling Laws for Routed Language Models. *ICML*, 2022. [arXiv:2202.01169](#).
- [29] Y. Tay, D. Bahri, L. Yang, D. Metzler, and D.-C. Juan. Sparse Sinkhorn Attention. *ICML*, 2020. [arXiv:2002.11296](#).
- [30] M. Caron, I. Misra, J. Mairal, P. Goyal, P. Bojanowski, and A. Joulin. Unsupervised Learning of Visual Features by Contrasting Cluster Assignments. *NeurIPS*, 2020. [arXiv:2006.09882](#).
- [31] Y. M. Asano, C. Rupprecht, and A. Vedaldi. Self-labelling via Simultaneous Clustering and Representation Learning. *ICLR*, 2020. [arXiv:1911.05371](#).
- [32] R. Wang, J. Yan, and X. Yang. Learning Combinatorial Embedding Networks for Deep Graph Matching. *ICCV*, 2019. [arXiv:1904.00597](#).
- [33] N. Courty, R. Flamary, D. Tuia, and A. Rakotomamonjy. Optimal Transport for Domain Adaptation. *IEEE TPAMI*, 2017. [arXiv:1507.00504](#).
- [34] Z. Ge, S. Liu, Z. Li, O. Yoshie, and J. Sun. OTA: Optimal Transport Assignment for Object Detection. *CVPR*, 2021. [arXiv:2103.14259](#).

A The matrix-free barrier gradient

This appendix derives in full the differentiation of the barrier primitive of §3, and — the point of the construction — shows how the backward pass costs the *same* $\mathcal{O}((n+m)\log(n+m))$ as the forward by reusing one Laplace-kernel multiply, never forming the dense $m \times n$ Jacobian. This is the engineering reason the barrier map can be used as a training layer: the gradient is not a symbolic dense object that must later be approximated, but a compiled tensor program with the same scans as the forward. Throughout, b_i ($i = 1, \dots, m$) solves $F(b_i) = \tau_i$ with $F(b) = \sum_{j=1}^n w_j \sigma((b - x_j)/h)$, σ the Laplace CDF, $f = \sigma' = \frac{1}{2}e^{-|\cdot|}$ its density, and $\bar{b}_i = \partial L / \partial b_i$ the upstream cotangent.

A.1 The three Jacobians

Differentiate the defining identity $F(b_i; x, w) = \tau_i$. The partials of F are

$$\left. \frac{\partial F}{\partial b} \right|_{b_i} = \frac{d_i}{h}, \quad \frac{\partial F}{\partial x_j} = -\frac{w_j}{h} f\left(\frac{b_i - x_j}{h}\right), \quad \frac{\partial F}{\partial w_j} = \sigma\left(\frac{b_i - x_j}{h}\right), \quad d_i := \sum_l w_l f\left(\frac{b_i - x_l}{h}\right),$$

so the implicit-function theorem ($\partial_b F db_i + \partial_\theta F d\theta = d\tau_i$) gives

$$\frac{\partial b_i}{\partial x_j} = \frac{w_j f((b_i - x_j)/h)}{d_i}, \quad \frac{\partial b_i}{\partial \tau_i} = \frac{h}{d_i}, \quad \frac{\partial b_i}{\partial w_j} = -\frac{h \sigma((b_i - x_j)/h)}{d_i}. \quad (13)$$

(Working with $\log w$ for stability, $\partial b_i / \partial \log w_j = w_j \partial b_i / \partial w_j$; the softmax that produces w from logits, and the sigmoid that produces τ_i from its logit, are then ordinary autograd nodes outside the operator.)

A.2 The VJP reuses two transposed kernel multiplies

Let $g_i = \bar{b}_i / d_i$. Contracting (13) with $\bar{b}$ gives the reverse-mode gradients

$$\frac{\partial L}{\partial x_j} = w_j \sum_i f\left(\frac{b_i - x_j}{h}\right) g_i, \quad \frac{\partial L}{\partial \log w_j} = -h w_j \sum_i \sigma\left(\frac{b_i - x_j}{h}\right) g_i, \quad \frac{\partial L}{\partial \tau_i} = \frac{h \bar{b}_i}{d_i}. \quad (14)$$

Every nontrivial sum here is over the m barriers, evaluated at the n points (or, for d_i , the reverse). Writing $K_f[i, j] = f((b_i - x_j)/h)$ and $K_\sigma[i, j] = \sigma((b_i - x_j)/h)$, this is

$$\partial L / \partial x = w \odot (K_f^\top g), \quad \partial L / \partial \log w = -h w \odot (K_\sigma^\top g), \quad d = K_f w,$$

Algebraically these formulas contain $K_f^\top g$, $K_\sigma^\top g$, and $d = K_f w$. The forward inverse already computes and caches d , so the backward executes only the two transposed structured multiplies; no $m \times n$ object is ever stored.

The width derivative. Differentiating the same identity $F(b_i) = \tau_i$ in h gives a fourth Jacobian,

$$\frac{\partial b_i}{\partial h} = \frac{1}{h d_i} \sum_j w_j f\left(\frac{b_i - x_j}{h}\right) (b_i - x_j), \quad (15)$$

a $\frac{1}{h}$ -scaled first moment of the same kernel. Its VJP needs no new kernel multiply. With the already computed $\partial L / \partial x = w \odot (K_f^\top g)$, we obtain

$$\frac{\partial L}{\partial h} = \sum_i \bar{b}_i \frac{\partial b_i}{\partial h} = \frac{1}{h} \left(\langle b, \bar{b} \rangle - \langle x, \partial L / \partial x \rangle \right), \quad (16)$$

two inner products of vectors the backward already holds. The width is therefore a first-class, matrix-free gradient target alongside the points, weights and levels; combined with the reclaimed-mean parametrisation of (11) ($\log h = -\bar{\ell}$), it lets h train directly from the logits while preserving positivity and the conventional relation between larger logits and a harder strip.

A.3 Why the multiply is $\mathcal{O}((n+m)\log)$: the sorted exponential scan

Each multiply is the Laplace kernel K_f (and its CDF analogue K_σ) applied to a vector. Consider $y_j = \sum_i e^{-|x_j - b_i|/h} v_i$ (so $\frac{1}{2}y = K_f^\top v$). Splitting the absolute value by the sign of $x_j - b_i$,

$$y_j = e^{-x_j/h} \underbrace{\sum_{b_i \leq x_j} e^{b_i/h} v_i}_{\text{left}} + e^{x_j/h} \underbrace{\sum_{b_i > x_j} e^{-b_i/h} v_i}_{\text{right}}. \quad (17)$$

Sorting the m anchors b once, the two inner sums are a prefix and a suffix scan of $e^{\pm b_i/h} v_i$; a single **searchsorted** of the n queries x_j into the sorted anchors then reads off, for every j , the prefix/suffix value at its position. Total work $\mathcal{O}((n+m)\log(n+m))$ (the sort and the binary searches), with $\mathcal{O}(n+m)$ memory — the dense kernel is never formed. In practice the exponentials are anchored to the local query point ($e^{(b_i - x_j)/h} \leq 1$ on the left, $e^{(x_j - b_i)/h} \leq 1$ on the right) so no term overflows; the production kernel evaluates (17) by the decoupled-lookback scan of §2.2, and the CDF multiply $\sum_i \sigma((b_i - x_j)/h) v_i$ adds only a plain suffix count $\sum_{b_i > x_j} v_i$ to the same two scans. This is the $\ker(a, b)$ multiply re-used verbatim from the forward operator: *the layer both is the reduction and differentiates it with one and the same primitive.*

A.4 The density d_i for free

The factor d_i appears in all three gradients through $g = \bar{b}/d$. Computing it by a kernel multiply $d = \frac{1}{2} K_f w$ would add one more $\mathcal{O}((n+m)\log)$ pass. It is instead read off the forward inverse at no extra cost: the barrier b_i lands in an interval $[x_l, x_r)$ of the sorted points, and the prefix/suffix log-masses λ_L, λ_R gathered there for the inverse solver are exactly the left/right kernel sums, so

$$d_i = \frac{1}{2} \left(e^{\lambda_L + (x_l - b_i)/h} + e^{\lambda_R + (b_i - x_r)/h} \right), \quad (18)$$

with both exponents ≤ 0 (since $x_l \leq b_i < x_r$) and the $-\infty$ end sentinels giving zero mass past the extreme points. The backward then performs only the two b -anchored multiplies of (14) (m small), and the single expensive $\mathcal{O}(n \log n)$ sort happens once, in the forward.

B Recap: the Fast LapSum sorted-scan primitives

The main text uses two primitives from Fast LapSum [5] as black boxes: the $\mathcal{O}(n \log n)$ sorted-scan threshold solve and the matrix-free Laplace-kernel multiply. We recap them here. For scores $r \in \mathbb{R}^n$, budget $k \in (0, n)$ and temperature $\alpha \neq 0$ (the main text's width h up to sign: $h = |\alpha|$, and $\alpha < 0$

orients larger scores toward larger mass, matching the increasing CDF F of the body), LapSum returns $p_i = \sigma((b - r_i)/\alpha)$ with the Laplace CDF

$$\sigma(t) = \frac{1}{2}e^t \quad (t \leq 0), \quad \sigma(t) = 1 - \frac{1}{2}e^{-t} \quad (t > 0), \quad (19)$$

where the threshold b is the unique solution of

$$\text{LapSum}(b) := \sum_{i=1}^n \sigma((b - r_i)/\alpha) = k. \quad (20)$$

From bisection to one bidirectional scan. The reference solve treats (20) as a black-box equation and runs bisection. Each bisection level evaluates $\text{LapSum}(\cdot)$ by summing over all n entries, so the same vector is repeatedly scanned. Sorting reveals a cheaper structure: as the threshold moves from one sorted score to the next, the left and right exponential tails only need to be rescaled. Thus the whole curve $\text{LapSum}(\cdot)$ at all sorted nodes can be computed by one forward scan and one backward scan.

Let $r_{(1)} \geq \dots \geq r_{(n)}$ be the sorted inputs (for $\alpha < 0$). Define the prefix/suffix exponential scans

$$P_i = \sum_{j \leq i} \exp\left(\frac{r_{(j)} - r_{(i)}}{\alpha}\right), \quad S_i = \sum_{j \geq i} \exp\left(\frac{r_{(i)} - r_{(j)}}{\alpha}\right). \quad (21)$$

Because $\alpha < 0$ and the differences are non-negative, every exponent is non-positive and the recurrences

$$P_i = 1 + e^{(r_{(i-1)} - r_{(i)})/\alpha} P_{i-1}, \quad S_i = 1 + e^{(r_{(i)} - r_{(i+1)})/\alpha} S_{i+1}$$

are unconditionally stable, no log-sum-exp shifting required.

Proposition 2. *For each sorted node, with the scores 1-indexed ($i = 1, \dots, n$),*

$$\text{LapSum}(r_{(i)}) = i - \frac{1}{2} + \frac{1}{2}(S_i - P_i), \quad (22)$$

so each node value is its half-integer reference $i - \frac{1}{2}$ plus the explicit tail correction $\frac{1}{2}(S_i - P_i)$. The correction vanishes in the well-separated limit; the node values do not, in general, lie exactly on the half-integer ladder at finite temperature.

Proof. Substitute (19) into (20) at $b = r_{(i)}$ and split the sum into $j < i$, $j = i$, $j > i$. The first group contributes

$$i - 1 - \frac{1}{2} \sum_{j < i} e^{(r_{(j)} - r_{(i)})/\alpha} = i - 1 - \frac{1}{2}(P_i - 1).$$

The middle term is $\frac{1}{2}$, and the last group is $\frac{1}{2} \sum_{j > i} e^{(r_{(i)} - r_{(j)})/\alpha} = \frac{1}{2}(S_i - 1)$. Summing gives (22). $\square$

Since $\text{LapSum}(r_{(i)})$ is monotone in i , binary search over (22) locates the interval containing b with $\mathcal{O}(\log n)$ lookups into P, S . Write this interval as $[r_{(\ell+1)}, r_{(\ell)}]$, so ℓ entries are strictly above it. Inside that interval the threshold solves

$$\frac{S_{\ell+1}}{2} e^{(b - r_{(\ell+1)})/\alpha} - \frac{P_\ell}{2} e^{(r_{(\ell)} - b)/\alpha} + \ell = k,$$

a quadratic in $u = e^{(b - r_{(\ell+1)})/\alpha}$ with closed-form root. Boundary cases ($k \leq \text{LapSum}(r_{(1)})$ or $k > \text{LapSum}(r_{(n)})$) reduce to a single exponential. Total work per batch element is $\mathcal{O}(n)$ (one scan) plus $\mathcal{O}(\log n)$ (the search), versus $\mathcal{O}(n \log n)$ for the reference implementation.

Implementation and backward. After sorting, the threshold solve and the final CDF evaluation are fused in a CUDA kernel. The prefix recurrence $P_i = w_i P_{i-1} + 1$ is represented by the associative transition matrix

$$\mathbf{M}_i = \begin{pmatrix} w_i & 1 \\ 0 & 1 \end{pmatrix},$$

so a block scan materialises all node values. The backward pass uses the implicit VJP

$$(\nabla_r p)^\top g = f \odot (g - \langle g, q \rangle), \quad f_i = \frac{1}{2|\alpha|} e^{-|b - r_i|/|\alpha|}, \quad q_i = \frac{f_i}{\sum_j f_j},$$

needing only element-wise work plus one reduction — the exact analogue of the softmax backward, whose vector-Jacobian product $s \odot (g - \langle g, s \rangle)$ is likewise a rank-one correction computed without the dense Jacobian. The matrix-free Laplace-kernel multiply $\sum_i e^{-|a - b_i|/h} v_i$ (and its CDF analogue), used by every gradient in the main text, is evaluated by the same prefix/suffix scan in $\mathcal{O}((n + m) \log(n + m))$ (the derivation is in §A).

C Order operators: recovering the LapSum family

Read at the appropriate weights and levels, the barrier map of §3 reproduces LapSum’s differentiable order operators: each is a different *reading* of the same inverse-CDF queries on one sorted pass, so they stop being separate layers. We record the correspondence here for completeness, together with the two limit facts (Propositions 3–4 and Theorem 3) that the main text cites. We build the operators up from the simplest, taking uniform weights $w_j = 1/n$ unless stated otherwise.

Soft order statistics and soft rank. Query the CDF at the $m = n$ midpoint levels $\tau_i = (i - \frac{1}{2})/n$, and write $o_i = F^{-1}((i - \frac{1}{2})/n)$. These midpoint quantiles are the *soft order statistics*: as $h \rightarrow 0$ the i -th o_i converges to the i -th smallest score, so $(o_1, \dots, o_n)$ is the softly sorted sequence. Dually, the *soft rank* of a point is the (scaled) mass lying below it,

$$\text{rank}_h(x_j) = \frac{1}{2} + n F(x_j) = \frac{1}{2} + \sum_{l=1}^n \sigma\left(\frac{x_j - x_l}{h}\right), \quad (23)$$

which tends to the 1-indexed integer position of x_j as $h \rightarrow 0$; the added $\frac{1}{2}$ corrects the self-mass $\sigma(0) = \frac{1}{2}$. Both are read off the single sort and the inverse-CDF queries of §3, with no $n \times n$ array — unlike the $\mathcal{O}(n^2)$ sorts NeuralSort and SoftSort build.

The soft permutation matrix uses bin boundaries, not the midpoint quantiles. To obtain rank slots with exact masses, set $q_0 = -\infty$, $q_n = +\infty$, and $q_i = F^{-1}(i/n)$ for $i = 1, \dots, n-1$. Thus o_i locates the i -th soft order statistic, whereas q_i separates slots i and $i+1$. The soft membership of point j in slot i is the mass its Laplace bump places in $[q_{i-1}, q_i)$,

$$A_{ij} = \sigma\left(\frac{q_i - x_j}{h}\right) - \sigma\left(\frac{q_{i-1} - x_j}{h}\right) \approx \Pr[\text{rank}(x_j) = i]. \quad (24)$$

A is a soft assignment matrix: at uniform weights and equal-mass levels it is doubly stochastic (Proposition 3) and collapses to the hard sorting permutation as $h \rightarrow 0$. Weighting its columns by the point weights gives the coupling $\Pi_{ij} = w_j A_{ij}$. Neither is ever materialised: Av (or Πv) is one Laplace-CDF scan and an adjacent difference, $(\Pi v)_i = (K_\sigma(w \odot v))_i - (K_\sigma(w \odot v))_{i-1}$, evaluated at the boundaries q , so every operator below stays matrix-free.

Sort and rank are the two sides of A . Contracting A with the *values* sorts them — the i -th output is the expected value of the point at rank i ,

$$(\text{soft sort})_i = \sum_j A_{ij} x_j = (Ax)_i, \quad (25)$$

recovering the hard order statistics as $h \rightarrow 0$. Contracting A from the *other side* with the rank labels $(1, \dots, n)$ returns the expected rank of each point, $\sum_i i A_{ij}$ — the matrix-view counterpart of the soft rank (23), with which it agrees as $h \rightarrow 0$. One matrix, two contractions: Ax orders the values, this rank contraction scores the points. Three facts pin down exactly what A expresses.

Proposition 3 (Exact marginals). $\Pi \in U(\nu, w)$, the transportation polytope with row marginal $\nu_i = \tau_i - \tau_{i-1}$ and column marginal w . For arbitrary target bin masses, set $\tau_0 = 0$, $\tau_m = 1$, $q_0 = -\infty$, $q_m = +\infty$, and $q_i = F^{-1}(\tau_i)$ internally. The columns telescope to $\sum_i \Pi_{ij} = w_j$, and because the internal boundaries solve $F(q_i) = \tau_i$ for the w -weighted CDF F , the rows give $\sum_j \Pi_{ij} = F(q_i) - F(q_{i-1}) = \nu_i$ exactly, for arbitrary weights w and level gaps ν . At uniform w and equal-mass levels ($m = n$, $\nu_i = 1/n$) A is doubly stochastic; arbitrary marginals give an exact-marginal coupling by a single strip — the same barrier mechanism that §4 pairs with a row-wise softmax and a diagonal to read out all of $U(s, t)$.

Proposition 4 (Smoothed bin coupling and zero-width limit). Let Z have the standard Laplace CDF σ . For every $h > 0$, $\Pi_{ij} = w_j \Pr[q_{i-1} < x_j + hZ \leq q_i]$ is the exact-marginal coupling obtained by assigning each source atom to a quantile bin after Laplace smoothing. It need not be the optimal monotone coupling of the original discrete atoms at finite h . As $h \rightarrow 0$ it converges to their one-dimensional monotone (quantile) coupling with target marginal ν . At equal mass with $m = n$ distinct points, $A \rightarrow P_\pi$ and the probability coupling $\Pi = A/n \rightarrow P_\pi/n$, where $\pi(j) = \text{rank}(x_j)$.

Theorem 3 (Universality through the coordinate). Fix uniform weights and equal-mass levels ($m = n$). Then

$$\{\lim_{h \rightarrow 0} A_h(x) : x \in \mathbb{R}^n \text{ distinct}\} = \{P_\pi : \pi \in S_n\},$$

the $n!$ permutation matrices — exactly the vertices of the Birkhoff polytope B_n . Every hard assignment is therefore the $h \rightarrow 0$ limit of the soft coupling for an appropriate coordinate, just as the temperature $\rightarrow 0$ limit of a differentiable sort recovers any permutation through the choice of scores.

operation	weights w_j	levels τ_i
soft top- k	uniform / logit	one, $\tau = (n-k)/n$
soft sort & rank	uniform	$(i-\frac{1}{2})/n$, $i=1:n$
soft permutation	uniform	equal-mass, $m=n$
soft reduction $M \rightarrow N$	logit (content)	equal-mass, $m=N-1$
weighted quantiles	logit	arbitrary τ_i

Table 13: One primitive (5), many operators: each is a choice of weights and target levels, computed by the same $\mathcal{O}((n+m)\log)$ inverse and differentiated by the same matrix-free VJP (6).

Proof sketch. By Proposition 4 every limit of A_h is the permutation that ranks x , so all limits are permutations; conversely, choosing x in the order prescribed by any $\pi \in S_n$ makes $\text{rank}(x_j) = \pi(j)$, realising P_π . Distinctness rules out ties and equal mass makes each $1/n$ -quantile bin hold exactly one order statistic in the limit. $\square$

These limits are the *vertices* (permutations) only: a doubly-stochastic matrix in the interior of B_n is not a single $h \rightarrow 0$ limit of A_h but is approximated at $h > 0$, along the low-dimensional family the $n-1$ barriers and the width h trace — not the full $(n-1)^2$ -dimensional polytope. And for a *fixed* coordinate the zero-width limit is always the monotone plan, never an arbitrary 2-D transport: the expressiveness comes from *learning* the coordinate x_j , not from solving a free coupling. This is exactly why the operator is closed-form and Sinkhorn-free — $\mathcal{O}((n+m)\log(n+m))$ with no iteration — rather than a general optimal-transport solver.

Soft selection and reduction. Selection is sorting with a budget. A single barrier ($m=1$) at the level $\tau = (n-k)/n$ is the soft top- k threshold, with keep-mass $p_j = 1 - \sigma((b-x_j)/h)$ summing to k (weighted points give budgeted selection under a prescribed importance). More generally $N-1$ equal-mass barriers split the points into N bins, and the column-weighted membership $\Pi = w \odot A$ aggregates each bin’s values into one summary — a differentiable reduction of M tokens to $N \ll M$ outputs, with logit weights making the cut *content-aware*. Selection is thus the $m < n$ instance of the same primitive.

Because the weights enter only through $\log w_j$ and the levels through their logits, each operation has a weighted form at no extra cost (Table 13): weighted soft-rank, weighted quantiles / quantile regression, and budgeted top- k are the same call with different (w, τ) , solved by one fast inverse and one matrix-free gradient.

D Proof of the column-strip lemma, and numerical validation

We fill in Lemma 1 — column-stochasticity, the marginal identity, and the $B_h \rightarrow I$ estimate behind the exact inverse of Theorem 2 — and record the numerical check. Recall the column grid $\eta_1 < \dots < \eta_m$ (minimal gap g), a mass $c \in \Delta^{m-1}$, and the strip $B_h(c)$ of (9). Its exterior cuts are $\beta_0 = -\infty$ and $\beta_m = +\infty$; only the internal cuts $\beta_j = F_Y^{-1}(T_j)$, $j = 1, \dots, m-1$, are solved, for $F_Y = \sum_j c_j \sigma(\frac{\cdot - \eta_j}{h})$.

Proof of Lemma 1. Column j' of $B_h(c)$ is the mass of a single Laplace bump at $\eta_{j'}$ distributed across the m strips $(\beta_{j-1}, \beta_j]$, so it sums to 1: $B_h(c)$ is column-stochastic. The marginal identity telescopes,

$$(B_h(c)c)_j = \sum_{j'} c_{j'} \left[\sigma\left(\frac{\beta_j - \eta_{j'}}{h}\right) - \sigma\left(\frac{\beta_{j-1} - \eta_{j'}}{h}\right) \right] = F_Y(\beta_j) - F_Y(\beta_{j-1}) = T_j - T_{j-1} = t_j.$$

For the estimate take $c = t$, put $t_{\min} = \min_j t_j$ and fix $\rho = g/4$. Choose $h_1 > 0$ small enough that

$$e^{-g/(2h_1)} \leq t_{\min}/2, \quad me^{-\rho/h_1} < 1.$$

For an interior cut $j = 1, \dots, m-1$, evaluate the CDF at $x_j^- := \eta_j + \rho$. The missing right tail of the atom at η_j is $\frac{1}{2}t_j e^{-\rho/h}$, whereas all leakage from centres to the right is at most $\frac{1}{2}e^{-(g-\rho)/h}$. Therefore

$$F_Y(x_j^-) - T_j \leq \frac{1}{2}e^{-\rho/h}(-t_j + e^{-(g-2\rho)/h}) < 0.$$

Likewise, at $x_j^+ := \eta_{j+1} - \rho$, the left tail of the atom at η_{j+1} dominates all leakage from centres to its left:

$$F_Y(x_j^+) - T_j \geq \frac{1}{2}e^{-\rho/h}(t_{j+1} - e^{-(g-2\rho)/h}) > 0.$$

Strict monotonicity now gives $\eta_j + \rho < \beta_j < \eta_{j+1} - \rho$. Consequently both boundaries of the j th strip are at least ρ from its centre η_j (with the two exterior boundaries at infinity), and the total mass of the j th Laplace bump outside its own strip is at most $e^{-\rho/h}$. Thus every entry of $B_h(t) - I$ has magnitude at most $e^{-\rho/h}$ and, for the induced infinity norm,

$$\|B_h(t) - I\|_\infty \leq m e^{-\rho/h}.$$

Our choice of h_1 makes this smaller than one, so $B_h(t)$ is invertible by the Neumann series. This proves the lemma with the fixed constants $\rho = g/4$ and $C = m$; only the admissible threshold h_1 deteriorates as $t_{\min} \downarrow 0$. $\square$

Numerical validation. A direct check (random interior Π^* , $n = 6$) confirms the claims of Theorem 2 to machine precision: the marginal identity $B_h(t)t = t$ and the exact solve $\text{diag}(s)W(h)B_h(t)^\top = \Pi^*$ hold to $\sim 10^{-17}$; $\|B_h(t) - I\|$ decays from 1.9×10^{-1} at $h=0.3$ to 1.4×10^{-11} at $h=0.02$; and the de-blur $W(h) > 0$ below an explicit threshold h_0 (here ≈ 0.2 for a well-interior draw — at $h=0.3$ it already overshoots into negative weights, and for targets with a smaller least entry the admissible threshold decreases at the logarithmic rate described in Theorem 2).

E The reference implementation: one scan, one strip

The whole of LAPHORN reuses one primitive — a sorted, log-domain prefix/suffix scan — first to solve all column barriers in parallel and then to apply the resulting strip. We sketch the pure-PyTorch reference kernel and the tricks that keep it exact and, under `torch.compile`, fast; the reference needs no hand-written CUDA (the optional fused kernels of §5.3 are for GPUs the compiler cannot fuse). The implementation target is therefore deliberately ordinary: a user can put the layer inside a PyTorch model, compile it, and get an exact prescribed-marginal forward together with the analytic backward, rather than relying on a separate projection routine or a custom kernel for one fixed size.

The barrier solve. Given weights w and centres x , the CDF $F(b) = \sum_j w_j \sigma((b - x_j)/h)$ is evaluated at all sorted nodes by four log-domain scans — the prefix/suffix sums of $e^{\pm x_j/h} w_j$ and of w_j — which we *fuse into one* batched `logcumsumexp` over a stacked $(4, n)$ tensor. Because the node CDF ladder is monotone, each target level is placed by one `searchsorted`, and the barrier inside the located interval is closed-form (a ratio of two decayed exponentials, both with exponent ≤ 0 , so nothing overflows). The weighted density d_i that the gradient needs is read off the same interval state and cached, so the backward executes two transposed matrix-free kernel multiplies and never forms the $m \times n$ Jacobian (Appendix A).

The interval masses and the two public interfaces. The coupling is $\Pi = \text{diag}(s)W B^\top$ with $B_{jc} = \sigma((\beta_j - c)/h) - \sigma((\beta_{j-1} - c)/h)$. Two orientations of the same strip primitive are used. To return the full plan, `coupling` evaluates $BW^\top$, transposes, and row-scales, because $(BW^\top)^\top = WB^\top$; this costs $\mathcal{O}(nm)$ and never forms B . To return only the action, `apply` evaluates the transpose action $Z = B^\top X$ first, followed by $Y = \text{diag}(s)WZ$; neither B nor Π is constructed. The positive masses used to solve the barriers admit the log-domain `logcumsumexp` scans described above. Values in X may have either sign, so the transpose strip action uses ordinary signed prefix/suffix accumulations with the same exponential factors, rather than a positive-mass log-domain reduction. The dense $n \times m$ score/softmax matrix W remains part of both computations. Sinkhorn likewise has a matrix-free factorised read-out once u, v are known; the distinction is that obtaining them to a feasibility tolerance requires iterative normalisation, whereas this score-to-output path is fixed and non-iterative.

Grid tricks: no sort, no bisection, no log factor. For the directional layer the column centres are the integer grid $0, \dots, m-1$, already ordered. Two consequences: a `presorted` fast path skips the $\mathcal{O}(m \log m)$ sort in the barrier solve, and `searchsorted` into the grid becomes $\lfloor \beta \rfloor$, an $\mathcal{O}(m)$ index. The barriers then cost $\mathcal{O}(m)$ (linear scans only) and the whole map $\mathcal{O}(nm)$, with no logarithmic factor in the leading term.

Exactness and the gradient. Everything is carried in the log domain with exponents anchored to the local query, so no term overflows: the marginals hold to 10^{-15} in fp64, the scan-based and the explicit dense factorisation agree to 2×10^{-16} , and `gradcheck` passes through the scores, the widths and the marginals. The gradient is matrix-free by construction: plain reverse mode through the “fast”

scan differentiates the log-domain prefix/suffix operations at forward cost and never forms the $(nm)^2$ Jacobian — and, being ordinary autograd, it is transparent to the compiler; the barrier solve uses the explicit implicit-function VJP of [Appendix A](#), with no fixed-point adjoint.